Conversación con Claude — TPI MS2: pruebas (Fase 2)

Integrante: Cristian Morales
Herramienta: Claude (Anthropic), sesión de trabajo en claude.ai
Período: 19/10/2026 22:29 a 31/10/2026 20:34 (hora de Argentina) — 110 mensajes
Fuente: exportación oficial de datos de la cuenta (Settings → Privacy → Export data), conversación «Conftest construction»
Nota: se transcribe el texto visible de cada mensaje. Se omiten el razonamiento interno del modelo y el detalle técnico de las herramientas; en su lugar se indica la acción realizada (archivos enviados, comandos ejecutados, accesos al repositorio). El acceso al repositorio fue de solo lectura: la IA no hizo commits ni push. Las capturas de pantalla adjuntas no se reproducen; la evidencia del aislamiento de la base está en `docs/evidencias/fase2/cristian/base_aislada.png`. Los mensajes posteriores a la exportación no están incluidos.
Parte: 3 — Pruebas automatizadas de la parte asignada (estudiantes y responsables): base de datos aislada con `conftest.py`, criterio de línea de base, tests de los casos de la Fase 1 y documento de resultados. Viene de «cristian_claude_02_diagnostico.pdf».

Usuario (Cristian Morales) 19/10/2026 22:29

[Archivos adjuntos: cristian.md, main.py, cristian_claude_02_diagnostico.pdf, cristian_claude_01_preparacion.pdf]

Contexto: TPI de Metodología de Sistemas II (UTN). Grupo: Gaston, Javier, Bruno y Cristian (yo).
Sistema base: Gestión de Solicitudes Académicas, FastAPI + SQLite, un solo main.py.
Repo: CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta. Ramas main y dev protegidas (PR con 1 aprobación); se trabaja en ramas desde dev.
Tags: estado-inicial (hecho), etapa-diagnostico, etapa-refactoring, etapa-pruebas, etapa-docs, estadorefactorizado.

Reparto:
- Gaston: registrar solicitud (líneas 114–198). Coordina tabla de smells. Garantiza Introduce Parameter Object.
- Javier: cambio de estado y asignación (251–318). Coordina code review y /docs. Garantiza Move Method.
- Bruno: búsqueda, detalle y reporte (201–248, 321–361). Coordina tests. Garantiza Extract Method.
- Cristian: conexión, tablas, datos de prueba, estudiantes y responsables (1–111). Coordina PDF, tags y video. Hace los cambios transversales.

Estado: terminé mi diagnóstico (Fase 1). 13 hallazgos, prioridad alta: 1 Global State, 2 efectos secundarios al importar, 3 manejo de errores inconsistente, 5 falta de modelo de entrada, 11 integridad de datos. PR abierto a dev, revisa Gaston.
Siguiente: Fase 2 (tests sobre estado-inicial, antes de refactorizar), después Fase 3 (resuelvo primero los transversales 1 y 2).

Quiero resolverlo con ayuda, no que me lo armes: guiame y revisá lo que hago.

entiendo que podemos armar los tests de todo sin necesidad de esperar los otros chicos no?

Claude 19/10/2026 22:30

Sí, pero con un matiz: podés arrancar **tus** tests sin esperar a nadie. No conviene que armes los tests **de todo**.

Por qué sí podés arrancar

Los tests no tocan `main.py`. Prueban los endpoints desde afuera, así que no generan conflictos con el diagnóstico de los demás ni dependen de él. Tus casos de la Fase 1 ya son prácticamente tu suite: los 12 casos pasados a pytest.

Por qué no "de todo"

1. **La participación.** Según el plan, cada uno escribe los tests de su parte. Si escribís los de Gaston, Javier y Bruno, en el repo quedan como tuyos y ellos pierden evidencia de la Fase 2. En la defensa, además, cada uno tiene que poder explicar sus tests.
2. **El `conftest.py` es de Bruno, y es lo primero que hay que resolver.** Si cada uno arma su propia forma de tener una base limpia, terminan con cuatro soluciones distintas. Acordalo con él antes de escribir tests en serio.

El punto más difícil, y te toca a vos entenderlo

Tu hallazgo 2 es justamente el obstáculo de los tests: `import main` crea `solicitudes.db` y carga los datos de prueba. Bruno tiene que lograr una base limpia por test **sin modificar** `main.py`. Para ayudarlo, pensá estas preguntas:

- La ruta en `sqlite3.connect("solicitudes.db", ...)` es relativa. ¿Relativa a qué?
- ¿En qué momento se ejecuta esa línea? ¿Una vez por test o una vez por proceso?
- pytest tiene `tmp_path` y `monkeypatch.chdir`. ¿Qué podrías combinar con eso, y con cuándo se importa `main`, para que cada test arranque con una base nueva?

Pasale tu respuesta a Bruno, o resuélvanlo juntos. Así además entendés a fondo el transversal que después te toca refactorizar.

Una decisión de criterio antes de escribir los asserts

Los tests de la Fase 2 sirven para demostrar que el refactoring **no cambió el comportamiento**. Entonces, ¿qué esperás en el caso 2 (falta apellido)? ¿El `200` que devuelve hoy o el `422` que debería devolver?

Pensalo en relación con tu hallazgo 3: pasar errores de 200 a 422 cambia lo que ve el cliente. ¿Eso es refactoring o es una corrección de defecto? La respuesta define cómo escribís los tests y cómo lo justifican en el informe. Es una buena discusión para tener con el grupo. Pista: en pytest existe `xfail`, y el caso 10 (el 500) es el candidato obvio.

Sobre el orden en el repo

Trabajá en tu rama (por ejemplo `test/personas`), pero no mergees a `dev` hasta que esté puesto `etapa-diagnostico`. Si no, el tag del diagnóstico queda sobre un commit que ya tiene tests, y se pierde la separación entre fases. El tag lo ponés vos, así que lo controlás.

Cuando tengas la idea de cómo resolver la base limpia, o el primer test escrito, pasámelo y lo revisamos.

Usuario (Cristian Morales) 19/10/2026 22:39

bien, estoy un poco confundido, osea podria hacer test de las suits que diagnostique, epro habria que ajustar lo de la db

Claude 19/10/2026 22:39

Sí, vas bien. Lo ordeno en dos piezas separadas:

1. La base limpia (infraestructura compartida)

Es el `conftest.py`. No prueba nada: prepara el terreno para que **todos** los tests arranquen igual. Es de Bruno, porque coordina tests, pero lo usan los cuatro.

El problema, en simple: hoy, si corrés los tests dos veces seguidas, la segunda vez el caso 1 (alta de `TUP2001`) falla con "legajo repetido", porque el estudiante quedó guardado en `solicitudes.db` de la corrida anterior. Los tests tienen que dar lo mismo siempre, sin importar qué pasó antes.

La restricción: hay que resolverlo sin tocar `main.py`, porque todavía estamos en la versión inicial.

2. Tus tests (lo tuyo)

Es `tests/test_personas.py`, con tus casos de la Fase 1 pasados a código. Cada caso es un test que manda el request y verifica el código HTTP y la respuesta.

Cómo seguir

1. **Hablá con Bruno** y preguntale si ya pensó lo de la base limpia. Si no, pensalo vos con las preguntas que te dejé antes: a qué carpeta es relativa la ruta `"solicitudes.db"` y cuándo se ejecuta esa línea. Lo mismo te toca entenderlo para tu refactoring de la Fase 3.
2. **Mientras tanto, escribí tus tests igual.** Arrancá con uno solo, el caso 1, usando `TestClient` de FastAPI. Corrélo dos veces seguidas: la segunda va a fallar. Esa es la prueba concreta de que hace falta la base limpia, y te sirve como evidencia.
3. **Antes de escribir los asserts, decidí qué esperás:** el comportamiento actual (200 con error) o el correcto (422). Es lo que te planteaba antes; si no lo tenés claro, lo charlamos.

Si querés, empezá por el paso 2: escribí el test del caso 1, pasámelo y lo revisamos antes de hacer los demás.

Usuario (Cristian Morales) 19/10/2026 22:43

```
def test_crear_estudiante_valido(client):
    datos = {
        "nombre": "Juan",
        "apellido": "Perez",
        "legajo": "TEST-9001",
        "email": "juan.perez@test.com",
        "carrera": "TUP"
    }

    respuesta = client.post("/estudiantes", json=datos)

    assert respuesta.status_code == 200

    resultado = respuesta.json()

    assert "id" in resultado
    assert resultado["nombre"] == "Juan"
    assert resultado["apellido"] == "Perez"
    assert resultado["legajo"] == "TEST-9001"
    assert resultado["email"] == "juan.perez@test.com"
    assert resultado["carrera"] == "TUP"
```

que te parece?

Claude 19/10/2026 22:43

Está bien encaminado: es claro, se lee solo y verifica todos los campos. Te marco cuatro cosas, de la más importante a la menos.

1. No prueba que se haya guardado (la más importante)

Fijate en el `return` de `crear_estudiante` (líneas 74–75): la respuesta se arma con los datos de `data`, no con lo que quedó en la base. Tu test pasaría aunque el `INSERT` no guardara nada.

Eso importa en la Fase 3: cuando muevas el SQL a un repositorio, si rompés el guardado, este test no lo detectaría. Para cubrirlo, ¿qué request podrías hacer después del POST para comprobar que el estudiante existe de verdad?

2. El 200 es una decisión, dejala escrita

`assert respuesta.status_code == 200` captura el comportamiento actual, aunque en tu tabla dijiste que debería ser 201. Está bien si es lo que decidieron, pero el que lea el test no lo sabe. Agregá un comentario corto que lo explique, por ejemplo que es el comportamiento de `estado-inicial` y que el correcto sería 201 según el hallazgo 3. Así nadie lo "arregla" sin querer y queda trazable.

3. `carrera: "TUP"` no prueba nada distinto

Mandás `"TUP"`, que es el mismo valor por defecto. Si el código ignorara tu dato, el test pasaría igual. Pensá dos tests en lugar de uno:

- uno que mande otra carrera y verifique que se respete;
- uno que **no** mande carrera y verifique que quede `"TUP"`.

4. Detalles

- **Trazabilidad:** un docstring como `"""Caso 1 de la Fase 1: alta válida."""` conecta el test con tu captura y tu tabla. Para el informe vale mucho.
- **`"id" in resultado`:** ¿alcanza con que exista, o también tendría que ser un número?

Antes de correrlo

¿De dónde sale `client`? Si todavía no existe el fixture en `conftest.py`, pytest va a fallar con *fixture 'client' not found*. Si lo armás vos de forma provisoria, acordalo con Bruno para no pisarse.

Ajustalo con estos puntos (sobre todo el 1) y pasámelo de nuevo. Después seguimos con un caso de error, que tiene sus propias decisiones.

Usuario (Cristian Morales) 19/10/2026 22:46

en base a lo que mencionas, se me ocurrio que podria separar en 2 el test, uno para verificar que se guarda en la base y respeta los datos enviados, y otro para comprobar el valor por defecto de carrera

```
def test_crear_estudiante_valido(client):
    """Caso 1 de la Fase 1: alta válida."""

    datos = {
        "nombre": "Juan",
        "apellido": "Perez",
        "legajo": "TEST-9001",
        "email": "juan.perez@test.com",
        "carrera": "Sistemas"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Se conserva el 200 de estado-inicial.
    # El código esperado para una creación sería 201 (hallazgo 3).
    assert respuesta.status_code == 200

    resultado = respuesta.json()

    assert isinstance(resultado["id"], int)
    assert resultado["nombre"] == "Juan"
    assert resultado["apellido"] == "Perez"
    assert resultado["legajo"] == "TEST-9001"
    assert resultado["email"] == "juan.perez@test.com"
    assert resultado["carrera"] == "Sistemas"

    # Comprobamos que el estudiante se haya guardado realmente.
    consulta = client.get("/estudiantes")
    estudiantes = consulta.json()

    guardado = next(
        (e for e in estudiantes if e["legajo"] == "TEST-9001"),
        None
    )

    assert guardado is not None
    assert guardado["nombre"] == "Juan"
    assert guardado["carrera"] == "Sistemas"


def test_crear_estudiante_sin_carrera_usa_default(client):
    """Si no se envía carrera, se utiliza TUP."""

    datos = {
        "nombre": "Ana",
        "apellido": "Lopez",
        "legajo": "TEST-9002",
        "email": "ana.lopez@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    assert respuesta.status_code == 200
    assert respuesta.json()["carrera"] == "TUP"
```

```
    # También verificamos el valor persistido.
    consulta = client.get("/estudiantes")
    estudiantes = consulta.json()

    guardado = next(
        (e for e in estudiantes if e["legajo"] == "TEST-9002"),
        None
    )

    assert guardado is not None
    assert guardado["carrera"] == "TUP"
```

Claude 19/10/2026 22:47

Muy bien. La separación en dos tests es correcta: cada uno prueba una sola cosa, y si falla, el nombre te dice qué se rompió. Los dos ajustes importantes están resueltos: ahora verificás que se guarde, y el `"Sistemas"` prueba que el dato enviado se respeta.

Quedan tres detalles, ninguno grave:

1. Buscá por `id`, no por `legajo`

Hoy buscás el registro guardado por legajo. Pero en tu diagnóstico (hallazgo 11) mostraste que el legajo **no es único** en la base. Si encontrás un registro con `TEST-9001`, no tenés garantía de que sea el que acabás de crear.

El `id` que devolvió el POST sí lo identifica sin ambigüedad. Buscar por `resultado["id"]` conecta la respuesta con lo guardado, que es justo lo que querés probar.

2. Verificá todos los campos guardados

En el primer test, de lo guardado solo comparás `nombre` y `carrera`. Si un refactor intercambiara `apellido` y `legajo` al insertar (el riesgo de tu hallazgo 8, los índices `r[2]`, `r[3]`), el test no lo vería. Hay una forma corta de comparar todo de una vez: pensá qué tendría que cumplir `guardado` respecto de `resultado`.

3. Código repetido en los tests

La búsqueda con `next(...)` está dos veces, y la vas a necesitar en varios tests más. Es el mismo Duplicated Code que marcaste en el sistema. Sacala a una función auxiliar, por ejemplo `buscar_estudiante(client, id)`, al principio del archivo. Además queda coherente con tu propio diagnóstico, lo que suma en la defensa.

Con eso, el alta válida queda cerrada. Lo siguiente es un **caso de error**, por ejemplo el caso 2 (sin apellido). Antes de escribirlo, pensá qué verificarías además del código y del mensaje. Pista: si el sistema dice "error", ¿qué no debería haber pasado en la base?

Y sigue pendiente lo de `client`: ¿ya hablaste con Bruno, o lo armás vos de forma provisoria?

Usuario (Cristian Morales) 20/10/2026 00:04

ahi ajuste por busqueda de ID y lo saque a una funcion auxiliar

```
def buscar_estudiante(client, estudiante_id):
    respuesta = client.get("/estudiantes")
    assert respuesta.status_code == 200

    return next(
        (e for e in respuesta.json() if e["id"] == estudiante_id),
        None
    )


def test_crear_estudiante_valido(client):
    """Caso 1 de la Fase 1: alta válida."""

    datos = {
        "nombre": "Juan",
        "apellido": "Perez",
```

```
        "legajo": "TEST-9001",
        "email": "juan.perez@test.com",
        "carrera": "Sistemas"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Se conserva el 200 de estado-inicial; el esperado sería 201.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    assert isinstance(resultado["id"], int)

    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado == resultado


def test_crear_estudiante_sin_carrera_usa_default(client):
    """Sin carrera explícita, el sistema utiliza TUP."""

    datos = {
        "nombre": "Ana",
        "apellido": "Lopez",
        "legajo": "TEST-9002",
        "email": "ana.lopez@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    assert respuesta.status_code == 200

    resultado = respuesta.json()
    assert resultado["carrera"] == "TUP"

    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado == resultado


para el caso de caso de error, probe algo asi

def test_crear_estudiante_sin_apellido(client):
    """Caso 2 de la Fase 1: falta apellido."""

    datos = {
        "nombre": "Pedro",
        "legajo": "TEST-9003",
        "email": "pedro@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Se registra el comportamiento actual.
    assert respuesta.status_code == 200
    assert respuesta.json() == {"error": "falta apellido"}

    # La petición fallida no debe crear el estudiante.
```

```
    consulta = client.get("/estudiantes")
    estudiantes = consulta.json()

    assert all(
        e["legajo"] != "TEST-9003"
        for e in estudiantes
    )
```

que te parece?

coni respecto a lo de Bruno, no lo hable pero hagamos algo provisorio

Claude 20/10/2026 00:05

Los dos primeros tests

Están muy bien. `guardado == resultado` es la forma corta que buscaba: compara todos los campos de una vez, y si un refactor intercambia columnas, falla. La función auxiliar también quedó bien, incluido el `assert` del 200 adentro.

El caso de error

Va bien: probaste lo más importante, que un error no deje nada guardado. Dos ajustes:

1. **El comentario quedó a medias.** En el caso 1 dejaste escrito el código esperado, y acá no. Para que sea consistente: el actual es 200 y el esperado, según tu tabla, 422.
2. **La verificación de "no se creó" es más débil de lo que parece.** Buscás por `legajo != "TEST-9003"`. ¿Y si un refactor roto guardara el estudiante con el legajo vacío o en `null`? Tu test no lo vería. Hay una forma más robusta que no depende de ningún campo. Pista: ¿qué número tendría que ser igual antes y después del POST?

Si la resolvés, va a servir para todos los casos de error (3, 5, 7...), así que también conviene sacarla a una función auxiliar.

El `client` provisorio

Lo armás vos en `tests/conftest.py`. Te doy las piezas y las preguntas, y vos lo escribís:

Lo que necesita el fixture:

- `TestClient` de `fastapi.testclient`, creado con la `app` de `main`.
- Los fixtures `tmp_path` (una carpeta temporal nueva por test) y `monkeypatch` (con `monkeypatch.chdir(...)` cambiás la carpeta de trabajo solo durante el test).

Lo que tenés que resolver:

- Si cambiás de carpeta **después** de importar `main`, ¿sirve de algo? ¿Por qué el orden importa?
- Python importa un módulo **una sola vez por proceso**: el segundo `import main` no vuelve a ejecutar nada. ¿Cómo lográs que cada test ejecute de nuevo las líneas 7 a 54? Buscá `sys.modules` o `importlib.reload`.

Para comprobar que funciona: corré `pytest` dos veces seguidas y las dos tienen que dar verde. Después fijate que en la raíz del repo no se haya modificado `solicitudes.db`.

Para no pisarte con Bruno: avisale hoy que armaste un `conftest.py` provisorio y en qué rama está. Si él ya estaba haciendo uno, que elijan juntos cuál queda. Y dejá un comentario arriba del archivo que diga que es provisorio.

Si `pytest` no encuentra `main` al importarlo, pasame el error: es un tema de rutas y se resuelve rápido.

Pasame el `conftest.py` cuando lo tengas, aunque no funcione.

Usuario (Cristian Morales) 20/10/2026 00:08

```
def buscar_estudiante(client, estudiante_id):
    """Busca un estudiante por su ID y devuelve el registro completo."""
    respuesta = client.get("/estudiantes")
    assert respuesta.status_code == 200

    return next(
        (e for e in respuesta.json() if e["id"] == estudiante_id),
```

```
        None
    )


def cantidad_estudiantes(client):
    """Devuelve la cantidad de estudiantes registrados."""
    respuesta = client.get("/estudiantes")
    assert respuesta.status_code == 200
    return len(respuesta.json())


def test_crear_estudiante_valido(client):
    """Caso 1: alta válida."""
    datos = {
        "nombre": "Juan",
        "apellido": "Perez",
        "legajo": "TEST-9001",
        "email": "juan.perez@test.com",
        "carrera": "Sistemas"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200. Esperado según el diagnóstico: 201.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    assert isinstance(resultado["id"], int)

    guardado = buscar_estudiante(client, resultado["id"])
    assert guardado is not None
    assert guardado == resultado


def test_crear_estudiante_sin_carrera_usa_default(client):
    """Sin carrera explícita, el sistema utiliza TUP."""
    datos = {
        "nombre": "Ana",
        "apellido": "Lopez",
        "legajo": "TEST-9002",
        "email": "ana.lopez@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    assert respuesta.status_code == 200

    resultado = respuesta.json()
    assert resultado["carrera"] == "TUP"

    guardado = buscar_estudiante(client, resultado["id"])
    assert guardado is not None
    assert guardado == resultado


def test_crear_estudiante_sin_apellido(client):
    """Caso 2: falta apellido."""
    datos = {
        "nombre": "Pedro",
        "legajo": "TEST-9003",
        "email": "pedro@test.com"
    }

    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200. Esperado según el diagnóstico: 422.
    assert respuesta.status_code == 200
```

```
    assert respuesta.json() == {"error": "falta apellido"}

    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

Claude 20/10/2026 00:08

Quedó muy bien. Los tres tests son consistentes entre sí, `cantidad_antes == cantidad_despues` no depende de ningún campo, y las dos auxiliares dejan los tests cortos y legibles. No le cambiaría nada de fondo.

Un solo detalle: el segundo test es el único sin el comentario de "Actual: 200. Esperado: 201". Agregalo para que los tres sigan el mismo criterio.

Lo que sigue

1. El `conftest.py`, primero. Hasta que exista, ninguno de estos tests corre, y no sabés si funcionan. Escribilo con las piezas que te pasé antes y pasámelo, aunque falle.

2. Después, el resto de tus casos. El patrón ya lo tenés armado, pero hay dos casos con trampa. Pensalos antes de escribirlos:

- **Casos 4, 6, 8b y 11 (el sistema acepta lo que no debería).** Si conservás el comportamiento actual, el test verifica que el estudiante con `email: "@"` **se crea**. Es correcto para la Fase 2, pero leído sin contexto parece que estás avalando el defecto. ¿Cómo lo dejarías claro en el test? El comentario de "actual/esperado" es un comienzo. Mirá también qué hace el marcador `@pytest.mark.xfail` y si te sirve para algún caso.
- **Caso 10 (el 500).** Probalo y fijate qué pasa: no vas a recibir un 500. Antes de buscar en Google, mirá el traceback que te tire y pensá por qué el `TestClient` se comporta distinto que `/docs` en tu captura. Pista: revisá los parámetros que acepta `TestClient` al crearlo.

Mandame el `conftest.py` cuando lo tengas.

Usuario (Cristian Morales) 20/10/2026 00:10

ayudame a construir el conftest

Claude 20/10/2026 00:11

Vamos de a poco: armamos el esqueleto juntos y vos completás las líneas clave. Son tres pasos.

Paso 0: que pytest encuentre `main`

Los tests están en `tests/` y `main.py` en la raíz. Si corrés `pytest` así nomás, el `import main` puede fallar con `ModuleNotFoundError`. Para evitarlo, creá un `pytest.ini` en la raíz del repo:

```
[pytest]
pythonpath = .
testpaths = tests
```

`pythonpath = .` agrega la raíz del repo a las rutas de importación, y `testpaths` le dice a pytest dónde buscar los tests.

Paso 1: el esqueleto

Creá `tests/conftest.py` con esto y completá los cuatro huecos:

```
# PROVISORIO: armado por Cristian para la Fase 2.
# Coordinar con Bruno antes de mergear.

import sys
import pytest
from fastapi.testclient import TestClient


@pytest.fixture
def client(tmp_path, monkeypatch):
    # (1) Pararse en una carpeta temporal vacía
    ...

    # (2) Asegurar que main se importe de cero
```

```
    ...

    # (3) Importar main
    ...

    # (4) Entregar el cliente al test y, al terminar, limpiar
    ...
```

Paso 2: las pistas para cada hueco

(1) Usá `monkeypatch.chdir(...)` con la carpeta que te da `tmp_path`. Como la ruta `"solicitudes.db"` es relativa, la base se va a crear ahí y no en el repo. Lo mismo pasa con `notificaciones.log`. Al terminar el test, `monkeypatch` vuelve solo a la carpeta original.

(2) Python guarda los módulos ya importados en `sys.modules`, que es un diccionario. Si `"main"` ya está ahí, el `import` no vuelve a ejecutar nada y te quedás con la conexión del test anterior. ¿Qué operación de diccionario lo saca, sin que falle si la clave no existe?

(3) Un `import main` común. Al estar después de (1) y (2), ejecuta las líneas 7 a 54 de nuevo, en la carpeta temporal.

(4) En un fixture, `yield` entrega el valor al test, y lo que va después del `yield` se ejecuta cuando el test termina. Entregá un `TestClient` creado con `main.app`. Después del `yield`, cerrá la conexión de `main`. Mirá en las líneas 7 y 8 cómo se llama. En Windows esto importa: si queda abierta, el archivo de la base queda bloqueado.

Paso 3: comprobar que funciona

1. `pytest -v`: tus tres tests en verde.
2. `pytest -v` de nuevo: otra vez en verde. Si la segunda falla, la base no se está limpiando.
3. `git status`: `solicitudes.db` no tiene que aparecer modificado.

Si algo falla, pasame el código y el error completo.

Usuario (Cristian Morales) 20/10/2026 00:16

```
==================== test session starts ====================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0 -- C:\Users\crist\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 3 items

tests/test_estudiantes.py::test_crear_estudiante_valido PASSED [ 33%]
tests/test_estudiantes.py::test_crear_estudiante_sin_carrera_usa_default PASSED [ 66%]
tests/test_estudiantes.py::test_crear_estudiante_sin_apellido PASSED [100%]

==================== warnings summary ====================
..\..\..\..\..\..\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\testclient.py:1
  C:\Users\crist\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\fastapi\testclient.py:1:
StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

..\..\..\..\..\..\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\starlette\testclient.py:53
  C:\Users\crist\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\starlette\testclient.py:53:
DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
```

entiendo que salio ok, dice 100% passed

Claude 20/10/2026 00:16

Sí, los tres pasaron. Una aclaración: el `[100%]` indica el **avance** de la corrida, no el porcentaje de aprobados. Lo que dice si pasaron es el `PASSED` de cada línea. Si alguno fallara, verías `FAILED` aunque la barra llegue a 100%.

Todavía no está comprobado del todo. Faltan tres cosas:

1. Terminar la verificación

Una sola corrida verde no prueba que la base se limpie. Hacé los otros dos pasos:

- Corré `pytest -v` **otra vez**. Si la base no se limpiara, acá fallaría.
- Corré `git status`. `solicitudes.db` no tiene que aparecer modificado.

2. Ojo: no corriste en el `.venv`

Mirá la primera línea:

```
C:\Users\crist\AppData\Local\Python\pythoncore-3.14-64\python.exe
```

Es el Python global de tu máquina, no el del `.venv` del proyecto. Funcionó porque tenés pytest instalado globalmente, pero con versiones que no son las del `requirements-dev.txt`. Por eso aparecen esos warnings.

Corré los tests como lo va a hacer el resto del grupo:

```
.\run.bat test
```

o activando antes el entorno (`.venv\Scripts\Activate.ps1`) y después `pytest -v`. En la primera línea tendría que aparecer una ruta con `.venv`.

Fijate además qué versión de Python acordaron en el grupo. Si es 3.11 o 3.12 y vos tenés 3.14, puede haber diferencias.

3. Los warnings

No vienen de su código: son avisos internos entre librerías (FastAPI, Starlette, anyio) por las versiones instaladas. No rompen nada. **No instales `httpx2`** porque lo diga el aviso: las dependencias se cambian en `requirements-dev.txt`, con acuerdo del grupo. Primero fijate si siguen apareciendo corriendo en el `.venv`.

Cuando tengas las dos corridas en verde dentro del `.venv`, pasame el `conftest.py` así lo reviso. Que los tests pasen no significa que esté bien armado: quiero ver el orden de los pasos y la limpieza después del `yield`.

Usuario (Cristian Morales) 20/10/2026 00:18

tire .\run.bat test ysalio esto

.\run.bat test
==================== test session starts ====================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 3 items

tests\test_estudiantes.py ... [100%]

==================== warnings summary ====================
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============== 3 passed, 1 warning in 0.10s ==============

Claude 20/10/2026 00:18

Ahora sí: corrió en el `.venv` (lo ves en la ruta del warning) y dio `3 passed`. Quedó un solo warning en lugar de dos.

Sobre el warning que queda

Sale de la combinación de versiones de Starlette y httpx que instaló pip. No tiene que ver con su código y no afecta los tests. Las opciones son:

- **Dejarlo.** Es lo más simple y no pasa nada.
- **Fijar versiones** en `requirements.txt` y `requirements-dev.txt` para que todos tengan exactamente las mismas. Eso se decide en grupo.

Lo que no conviene es ocultarlo con un filtro en `pytest.ini` sin acordarlo: es un aviso real de las dependencias, y si alguien después lo busca, no lo va a encontrar.

Un detalle de la versión de Python

El `.venv` también usa Python 3.14.7, porque se creó con el Python de tu máquina. Confirmá con el grupo qué versión acordaron. Si los demás usan 3.12, por ejemplo, conviene que todos usen la misma antes de que los tests den distinto en cada máquina.

Lo que falta

1. Correr `.\run.bat test` **una vez más** y confirmar que sigue en verde. Esta es la prueba real de que la base se limpia.
2. `git status`: `solicitudes.db` no tiene que aparecer modificado.
3. Pasarme el `conftest.py` para revisarlo.

Usuario (Cristian Morales) 20/10/2026 00:20

.\run.bat test ========== test session starts ========== platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0 rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta configfile: pytest.ini testpaths: tests plugins: anyio-4.15.1 collected 3 items

tests\test_estudiantes.py ... [100%]

========== warnings summary ========== .venv\Lib\site-packages\fastapi\testclient.py:1 C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead. from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html ========== 3 passed, 1 warning in 0.10s ========== git status solicitudes.db On branch docs/diagnostico-cristian Your branch is up to date with 'origin/docs/diagnostico-cristian'.

nothing to commit, working tree clean

aca te paso el conftest

```
# PROVISORIO: armado por Cristian para la Fase 2.
# Coordinar con Bruno antes de mergear.

import sys
import importlib
from pathlib import Path

import pytest
from fastapi.testclient import TestClient


@pytest.fixture
def client(tmp_path, monkeypatch):
    # (1) Pararse en una carpeta temporal vacía
    raiz_proyecto = Path(__file__).resolve().parents[1]
    if str(raiz_proyecto) not in sys.path:
        sys.path.insert(0, str(raiz_proyecto))

    monkeypatch.chdir(tmp_path)
```

```
    # (2) Asegurar que main se importe de cero
    sys.modules.pop("main", None)

    # (3) Importar main
    main = importlib.import_module("main")

    # (4) Entregar el cliente al test y, al terminar, limpiar
    try:
        with TestClient(main.app) as test_client:
            yield test_client
    finally:
        main.conn.close()
        sys.modules.pop("main", None)
```

Claude 20/10/2026 00:20

El `conftest.py` está bien resuelto: el orden es el correcto (primero `chdir`, después sacar `main` de `sys.modules`, después importar), y la limpieza con `try/finally` asegura que la conexión se cierre aunque el test falle. Usar `with TestClient(...)` también está bien. Antes de los ajustes, hay dos cosas más importantes que el código.

1. Mi verificación con `git status` no servía

Te pedí `git status` para comprobar que `solicitudes.db` no cambiara, pero seguramente el `.gitignore` del proyecto ignora los `.db`. Si es así, `git status` nunca lo va a mostrar, cambie o no. Ese "working tree clean" no prueba nada. Fue un error mío.

Una verificación que sí sirve, en PowerShell desde la raíz del repo:

```
(Get-Item solicitudes.db).LastWriteTime
.\run.bat test
(Get-Item solicitudes.db).LastWriteTime
```

Si las dos fechas son iguales, los tests no tocaron la base real. Si te dice que el archivo no existe, mejor todavía: ningún test lo creó.

2. Estás en la rama equivocada

`git status` dice `On branch docs/diagnostico-cristian`. Esa es la rama de tu PR de diagnóstico, que está esperando la revisión de Gaston. Si commiteás los tests ahí, entran en ese PR y se mezclan las fases.

Como los archivos nuevos todavía no están commiteados, viajan con vos al cambiar de rama:

```
git checkout dev
git pull
git checkout -b test/personas
```

Fijate con `git status` que aparezcan `tests/` y `pytest.ini` como archivos nuevos, y recién ahí commiteá.

3. Ajustes al `conftest.py`

- **El bloque de `sys.path` sobra.** Hace lo mismo que `pythonpath = .` en el `pytest.ini`. Tener dos mecanismos para lo mismo confunde a quien lo lea. Dejá solo el del `pytest.ini`, que es el lugar estándar, y sacá el bloque y el `import Path` si ya no lo usás.
- **El comentario (1) quedó desfasado.** Dice "pararse en una carpeta temporal", pero lo primero que hace el bloque es tocar `sys.path`. Al sacar ese bloque, se arregla solo.
- **Un docstring en el fixture.** Bruno y los demás lo van a usar sin haberlo escrito. Una línea que diga qué garantiza (una base nueva en una carpeta temporal por cada test) les ahorra leer el código.

Con estos ajustes, la base de tests queda lista. Hoy mismo avisale a Bruno: pasale el `conftest.py` y explicale por qué hace falta cada paso. Esa explicación es justamente tu hallazgo 2, así que también te sirve como práctica para la defensa.

Después seguimos con el resto de tus casos, empezando por el 10, que tiene la sorpresa del `TestClient`.

Usuario (Cristian Morales) 20/10/2026 00:21

si, lo vi lo de la rama, en breve la cambio te paso lo de l checkeo, veo que no se graba en la BD, asi que estamos ok
(Get-Item solicitudes.db).LastWriteTime

lunes, 19 de octubre de 2026 09:28:38 p. m.

.\run.bat test
========== test session starts ==========
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 3 items

tests\test_estudiantes.py ...   [100%]

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 3 passed, 1 warning in 0.10s ==========
(Get-Item solicitudes.db).LastWriteTime

lunes, 19 de octubre de 2026 09:28:38 p. m.

Claude 20/10/2026 00:21

Confirmado: la fecha es la misma antes y después (21:28, de cuando levantaste el sistema para las capturas de la Fase 1). Los tests trabajan en su propia base temporal y no tocan la real. El `conftest.py` cumple su función.

Te conviene guardar esa salida (fecha, tests, fecha) como captura, por ejemplo `docs/evidencias/fase2/cristian/base_aislada.png`. Prueba algo que el informe tiene que justificar: que los tests son repetibles y no dependen de corridas anteriores.

Lo que te queda, en orden:

1. Pasar los archivos a la rama `test/personas`.
2. Aplicar los ajustes al `conftest.py`: sacar el bloque de `sys.path` y agregar el docstring.
3. Avisarle a Bruno y pasarle el `conftest.py`.
4. Seguir con el caso 10.

Cuando tengas el caso 10, aunque falle de una forma que no esperabas, pasame el test y el error.

Usuario (Cristian Morales) 20/10/2026 00:22

[1 captura de pantalla adjunta]

te dejo la captura, para guardarla

Claude 20/10/2026 00:22

▸ *Buscó archivos de imagen*

La captura sirve: se ve la fecha antes, la corrida con `3 passed` y la fecha después, todo en una sola imagen y en orden. Es justo lo que hace falta.

No puedo guardarla como archivo desde acá: la imagen me llega para verla, pero no como archivo que pueda copiar al zip. Guardala vos directamente en el repo:

```
docs/evidencias/fase2/cristian/base_aislada.png
```

Commiteala junto con los tests en `test/personas`, no en la rama del diagnóstico: en el prompt se ve que todavía estás en `docs/...`.

Con eso, la base de los tests queda cerrada y documentada. Seguimos con el caso 10 cuando lo tengas.

Usuario (Cristian Morales) 20/10/2026 00:23

[1 captura de pantalla adjunta]

ya me pase de rama

Claude 20/10/2026 00:23

Bien, los archivos viajaron con vos. Antes de commitear, confirmá una cosa: en la captura se ve el `checkout -b`, pero no si lo hiciste parado en `dev`. Si la rama salió de `docs/diagnostico-cristian`, arrastra los commits del diagnóstico, y tu PR de tests los va a incluir.

Para comprobarlo:

```
git log --oneline dev..test/personas
```

Si no muestra nada, la rama salió de `dev` y está bien. Si aparece tu commit del diagnóstico, avisame y lo corregimos antes de seguir: todavía no commiteaste nada, así que es fácil.

Si está bien, antes del commit:

1. **Aplicá los ajustes al `conftest.py`**: sacar el bloque de `sys.path` y agregar el docstring. Así el primer commit ya queda limpio.
2. **Guardá la captura** en `docs/evidencias/fase2/cristian/base_aislada.png`. Todavía no aparece en el `git status`.
3. Agregá los archivos y revisá qué quedó preparado:

`powershell git add pytest.ini tests/ docs/ git status`

Fijate que no se cuele ninguna carpeta `__pycache__`. 4. Commiteá con el prefijo acordado. Por ejemplo:

`powershell git commit -m "test: base aislada para tests y casos de alta de estudiantes"`

Podés hacer `push` para tener el respaldo en GitHub, pero **no abras el PR a `dev` todavía**. Primero tiene que estar puesto `etapa-diagnostico`, así el tag no queda sobre un commit con tests.

Usuario (Cristian Morales) 20/10/2026 00:25

```
# PROVISORIO: armado por Cristian para la Fase 2.
# Coordinar con Bruno antes de mergear.

import sys
import importlib

import pytest
from fastapi.testclient import TestClient


@pytest.fixture
def client(tmp_path, monkeypatch):
    """Garantiza una base de datos nueva en una carpeta temporal por test."""

    # (1) Pararse en una carpeta temporal vacía
    monkeypatch.chdir(tmp_path)

    # (2) Asegurar que main se importe de cero
    sys.modules.pop("main", None)

    # (3) Importar main
    main = importlib.import_module("main")

    # (4) Entregar el cliente al test y, al terminar, limpiar
    try:
        with TestClient(main.app) as test_client:
```

```
            yield test_client
    finally:
        main.conn.close()
        sys.modules.pop("main", None)
```

asi?

Claude 20/10/2026 00:25

Así está bien. Quedó corto, cada paso hace una sola cosa y el comentario coincide con el código.

Una sugerencia para el docstring: "base de datos nueva" puede hacer pensar que arranca **vacía**, pero no: como `main` se importa de cero, carga los datos de prueba de las líneas 21–54 (4 estudiantes, 5 responsables, 6 solicitudes). Bruno y los demás lo van a necesitar saber. Por ejemplo, si alguien espera que `GET /estudiantes` devuelva una lista vacía, el test le va a fallar. Agregá una línea que lo aclare.

Antes de commitear, corré `.\run.bat test` una vez más. Sacaste el bloque de `sys.path`, así que ahora el import depende solo del `pytest.ini`. Si da `3 passed`, confirma que alcanza.

Después seguí con los pasos de antes: verificar la rama, guardar la captura, `git add`, revisar el `git status` y commitear.

Usuario (Cristian Morales) 20/10/2026 00:27

pasame la captura

Claude 20/10/2026 00:27

No la tengo como archivo: las imágenes que mandás me llegan solo para verlas, así que no puedo devolvértela ni descargarla.

Tampoco te conviene que la "reconstruya": una captura armada por mí no sería evidencia real de que corriste los tests.

Tenés dos opciones:

1. **Recuperar la que ya sacaste.** Si la tomaste con la herramienta de recortes de Windows (`Win + Shift + S`), puede estar en `Imágenes\Capturas de pantalla`. Si la pegaste directo en el chat, quizás solo quedó en el portapapeles y ya se perdió.
2. **Sacarla de nuevo.** Es lo más simple y además queda mejor, porque ahora corresponde al `conftest.py` final:

```
powershell (Get-Item solicitudes.db).LastWriteTime .\run.bat test (Get-Item solicitudes.db).LastWriteTime
```

Capturá la terminal con `Win + Shift + S` y guardala como `docs/evidencias/fase2/cristian/base_aislada.png`.

Con la opción 2 matás dos pájaros de un tiro: verificás que el `pytest.ini` alcanza sin el bloque de `sys.path` y obtenés la evidencia actualizada.

Usuario (Cristian Morales) 20/10/2026 00:27

pero te mande la captura, no podes mandarla como arhcivo?

Claude 20/10/2026 00:27

▸ *Ejecutó un comando: Buscar la captura adjunta*

▸ *Ejecutó un comando: Copiar la captura con el nombre de evidencia*

▸ *Envió el archivo base_aislada.png*

Tenías razón y me equivoqué: la captura sí estaba disponible como archivo. Te la mandé con el nombre `base_aislada.png`. Guardala en `docs/evidencias/fase2/cristian/`.

Un detalle: es de antes de que sacaras el bloque de `sys.path` del `conftest.py`. Sirve igual como evidencia de que la base queda aislada. Igual, antes de commitear, corré `.\run.bat test` una vez para confirmar que sigue en verde solo con el `pytest.ini`. Si querés que la evidencia corresponda exactamente al `conftest.py` final, reemplazá esta captura por la de esa corrida.

Usuario (Cristian Morales) 20/10/2026 00:28

[1 captura de pantalla adjunta]

ahi corrio super

Claude 20/10/2026 00:28

Perfecto: `3 passed` con el `conftest.py` final, en la rama `test/...` (se ve en el prompt). Queda confirmado que el `pytest.ini` alcanza solo, sin el bloque de `sys.path`.

Para la evidencia, quedate con la captura anterior (`base_aislada.png`). Esta no muestra las fechas de `solicitudes.db`, que es lo que prueba el aislamiento.

Ahora podés commitear:

```
git log --oneline dev..test/personas
git add pytest.ini tests/ docs/
git status
git commit -m "test: base aislada para tests y casos de alta de estudiantes"
git push -u origin test/personas
```

El primer comando es la verificación pendiente de que la rama salió de `dev`: no tiene que mostrar nada. Y recordá: sin PR hasta que esté el tag `etapa-diagnostico`.

Después, avisale a Bruno lo del `conftest.py` y seguimos con el caso 10.

Usuario (Cristian Morales) 20/10/2026 10:04

perfecto sigamos con el caso 10

Claude 20/10/2026 10:05

El caso 10 es el del `email: null`, el que te dio 500 en la Fase 1. Tiene dos partes: primero ver qué pasa, después decidir qué esperar.

Paso 1: escribilo como si fuera un caso más

Con el mismo patrón que los anteriores: armá los datos con `"email": None`, hacé el POST y escribí `assert respuesta.status_code == 500`. Corrélo.

Antes de correrlo, mirá la línea 67 de `main.py` y anotá qué esperás que pase: ¿qué hace `"@" not in None`?

Paso 2: leé el resultado con cuidado

No te va a dar ni `PASSED` ni un `FAILED` por el assert. Vas a ver un error en otra línea. Antes de pasármelo, fijate en el traceback:

- ¿En qué línea del test falla? ¿Llega al `assert`?
- ¿Qué tipo de excepción aparece?
- En `/docs` recibiste un 500. ¿Por qué acá no?

La pista: `TestClient` tiene un parámetro que decide qué hacer cuando la app lanza una excepción no manejada. Buscalo en la documentación de Starlette.

Paso 3: la decisión (la parte importante)

Hay dos formas válidas de escribir este test, y elegir una es criterio tuyo:

- **A. Registrar el comportamiento actual:** el test verifica que hoy da 500.
- **B. Registrar el comportamiento esperado:** el test verifica el 422 que debería dar, marcado con `xfail` porque hoy no se cumple.

Preguntas para decidir:

1. Los otros casos los escribiste con el comportamiento actual (200 con error). ¿Este es distinto? Pensá en la diferencia entre "responde mal" y "se rompe".
2. Si en la Fase 3 alguien arregla el 500, ¿qué tendría que pasar con este test en cada opción? ¿Cuál de las dos te avisa?
3. En `xfail` existe `strict=True`. ¿Qué cambia?

Si elegís A, además tenés que resolver cómo hacer que el `client` no lance la excepción. Pista: no conviene cambiar el fixture para todos, porque los demás tests dejarían de mostrarte errores inesperados.

Pasame el traceback del paso 2 y la opción que elegiste con su porqué. El código lo vemos después.

Usuario (Cristian Morales) 20/10/2026 10:16

Elegíque se vea el comportamiento actual del sistema porque en esta fase buscamos establecer una línea de base antes del refactor.

```
def test_crear_estudiante_email_null(client):
    """Caso 10: email nulo provoca un error 500."""
    datos = {
        "nombre": "Pedro",
        "apellido": "Perez",
        "legajo": "TEST-9010",
        "email": None
    }

    # Cliente específico para observar el HTTP 500 sin relanzar
    # la excepción no controlada por la aplicación.
    from fastapi.testclient import TestClient
    import main

    with TestClient(
        main.app,
        raise_server_exceptions=False
    ) as cliente_sin_excepciones:
        respuesta = cliente_sin_excepciones.post(
            "/estudiantes",
            json=datos
        )

    # Comportamiento actual: 500.
    # Comportamiento esperado tras corregir el defecto: 422.
    assert respuesta.status_code == 500
```

Claude 20/10/2026 10:17

La decisión está bien justificada: en la Fase 2 la suite es la línea de base, y un test que hoy pasa con el 500 es coherente con los demás. El comentario actual/esperado también está bien. Solo tené en cuenta la consecuencia: cuando en la Fase 3 corrijan el defecto, este test va a fallar **a propósito** y habrá que actualizarlo. Eso no es un problema, es justo la diferencia entre refactoring (no cambia comportamiento) y corrección de defecto. Anotalo para el informe.

`raise_server_exceptions=False` también es lo correcto, y armaste un cliente aparte en vez de cambiar el fixture para todos. Hay tres ajustes:

1. El `import main` es frágil (el más importante)

Funciona por casualidad. Como el test pide `client`, el fixture ya importó `main` en la carpeta temporal, y tu `import main` reutiliza ese módulo desde `sys.modules`. Ahora pensá: si alguien copia este test y se olvida de pedir `client`, ¿en qué carpeta se ejecuta la línea 7, y qué base abre?

No necesitás importar nada: el `client` que ya recibís conoce la app. Buscá qué atributo de `TestClient` la guarda.

Y si después aparecen más casos de 500 (a Gaston le puede pasar con solicitudes), conviene llevarlo a un fixture en `conftest.py`, por ejemplo `client_sin_excepciones`. Por ahora, con un solo caso, alcanza con resolverlo en el test.

2. Falta verificar que no se guardó nada

En el caso 2 comprobaste que un error no deja registros. Acá vale lo mismo, y más: hoy el crash ocurre en la línea 67, antes del `INSERT`, pero si un refactor mueve la validación del email después del guardado, tendrías un 500 **y** un estudiante a medio cargar. Ya tenés `cantidad_estudiantes` para eso.

3. Los imports van arriba

`from fastapi.testclient import TestClient` va al principio del archivo, como en cualquier módulo. Dentro de la función esconde una dependencia.

Pendiente del paso 2

No me pasaste el traceback. No lo necesito para revisar el código, pero en la defensa te van a preguntar por qué en `/docs` ves un 500 y en el test, sin ese parámetro, una excepción. ¿Podés explicarlo en dos líneas? Incluí

qué excepción salta y por qué la provoca la línea 67.

Aplicá los ajustes y pasame el test de nuevo junto con esa explicación.

Usuario (Cristian Morales) 20/10/2026 10:19

La línea 67 intenta evaluar "@" not in email, pero cuando email es None, Python lanza un TypeError porque None no es iterable. En /docs se observa la respuesta HTTP 500; en TestClient, por defecto, la excepción no controlada se vuelve a lanzar al test. Por eso usamos raise_server_exceptions=False para inspeccionar la respuesta 500.

Anotá que este test registra el comportamiento actual como línea de base. Cuando se corrija el defecto en la Fase 3 para devolver 422, el test fallará intencionalmente hasta que se actualice la expectativa. Eso distingue una corrección de defecto de un refactor que preserva el comportamiento.

Claude 20/10/2026 10:19

▸ *Guardó la anotación en sus notas de seguimiento*

Anotado: lo guardé en el seguimiento del TPI para el informe.

La explicación está completa: decís qué excepción salta, por qué la provoca la línea 67 y por qué `/docs` y `TestClient` se comportan distinto. Para la defensa, sumá un detalle: el 500 lo genera FastAPI cuando la excepción llega al servidor sin que nadie la maneje. No es un código que el sistema devuelva a propósito. Eso conecta con tu hallazgo 3 (manejo de errores inconsistente).

También conviene llevar esa anotación al informe, en la sección de criterios de la Fase 2. Mejor si va como criterio del grupo y no solo de tu parte, porque a Gaston y a Javier les va a pasar lo mismo con sus casos de 200 con error.

Falta el test corregido, con los tres ajustes:

1. Crear el cliente a partir del `client` que ya recibís, sin `import main`.
2. Comparar la cantidad de estudiantes antes y después.
3. Pasar el import de `TestClient` arriba de todo.

Pasámelo y lo cerramos.

Usuario (Cristian Morales) 20/10/2026 10:19

te paso el nuevo test

```
from fastapi.testclient import TestClient


def test_crear_estudiante_email_null(client):
    """Caso 10: email nulo provoca un error 500."""
    datos = {
        "nombre": "Pedro",
        "apellido": "Perez",
        "legajo": "TEST-9010",
        "email": None
    }

    cantidad_antes = cantidad_estudiantes(client)

    with TestClient(
        client.app,
        raise_server_exceptions=False
    ) as cliente_sin_excepciones:
        respuesta = cliente_sin_excepciones.post(
            "/estudiantes",
            json=datos
        )

    # Actual: 500. Esperado tras corregir el defecto: 422.
    assert respuesta.status_code == 500
```

```
    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

Claude 20/10/2026 10:19

Quedó bien. Los tres ajustes están aplicados:

- `client.app`: el cliente nuevo usa exactamente la misma app y la misma base temporal que el fixture, sin depender de un import.
- **Cantidad antes y después**: queda cubierto el riesgo de un 500 con un registro a medio cargar.
- **El import arriba**: la dependencia queda a la vista.

Además, el test es coherente con los anteriores: mismo patrón, mismo comentario actual/esperado y la misma verificación de que no se guardó nada.

Un detalle de forma: el import de `TestClient` va en la parte de arriba de `test_estudiantes.py`, junto con los demás imports y antes de las funciones auxiliares, no pegado a este test. Si ya lo pusiste ahí, está perfecto.

Corré `.\run.bat test`: tendrías que ver `4 passed`. Después commiteá en `test/personas`.

Lo que sigue: los casos que el sistema acepta y no debería

Son el 4, el 6, el 8b y el 11. Acá la trampa es al revés que en el caso 10: el test va a verificar que algo incorrecto **se guarda**. Antes de escribir el primero, pensá:

1. Con tu criterio de línea de base, ¿qué verificás? ¿Solo el 200, o también que el registro quedó guardado con el dato inválido?
2. ¿Cómo hacés para que alguien que lee el test sin contexto no crea que estás avalando el defecto? Mirá el nombre del test, no solo el comentario.
3. El caso 11 (legajo repetido o integridad de datos) es tu hallazgo de prioridad alta. ¿Qué necesitás crear **antes** del POST para poder reproducirlo?

Empezá por uno, el que te resulte más claro, y pasámelo.

Usuario (Cristian Morales) 20/10/2026 10:22

pasame para hacer el commit

Claude 20/10/2026 10:22

Antes de commitear, corré los tests para confirmar que el nuevo pasa:

```
.\run.bat test
```

Tiene que dar `4 passed`. Si da eso:

```
git status
git add tests/test_estudiantes.py
git commit -m "test: caso 10, email nulo provoca error 500 (linea de base)"
git push
```

En el `git status` tendría que aparecer solo `tests/test_estudiantes.py` como modificado. Si aparece algo más (por ejemplo un `__pycache__` o un `.db`), avisame antes de agregarlo.

Cuando esté pusheado, seguimos con los casos que el sistema acepta y no debería (4, 6, 8b y 11).

Usuario (Cristian Morales) 20/10/2026 10:23

listo

Claude 20/10/2026 10:23

▸ *Ejecutó un comando: Buscar los casos en el PDF del diagnóstico*

▸ *Ejecutó un comando: Leer la tabla de resultados y el detalle del caso 11*

Bien. Seguimos con los casos que el sistema acepta y no debería. Según tu tabla de la Fase 1, son estos:

Caso	Body	Hoy	Debería
4	`"email": "@"`	200, lo crea	422
6	`"apellido": null`	200, lo crea	422

Caso	Body	Hoy	Debería
8b	responsable repetido (igual al 8a)	200, crea un duplicado	409
11	`"legajo": null` dos veces	200, crea dos registros iguales	rechazo

Una corrección a lo que te dije antes: el caso 11 no necesita nada creado de antemano por fuera del test. Se reproduce con dos POST seguidos dentro del mismo test.

Empezá por el caso 4

Es el más simple y define el patrón para los otros tres. Antes de escribirlo, decidí:

1. **Qué verificás.** Con tu criterio de línea de base, no alcanza con el 200. ¿Qué tiene que quedar en la base? Ya tenés `buscar_estudiante` para comprobarlo.
2. **El nombre del test.** `test_crear_estudiante_email_invalido` no dice si se acepta o se rechaza. Elegí un nombre que diga lo que pasa **hoy**, para que nadie crea que es el comportamiento deseado.
3. **El comentario.** El mismo formato actual/esperado que venís usando.

Para después, dos cosas para ir pensando

- **El caso 8b es de responsables**, no de estudiantes. ¿Va en `test_estudiantes.py` o en otro archivo? ¿Te hace falta una auxiliar nueva?
- **El caso 11 es el más interesante para la defensa.** En la Fase 1 viste que el control de legajo repetido falla con `null` porque `legajo = NULL` nunca da verdadero en SQL. ¿Qué verificarías después del segundo POST para que el test muestre exactamente eso?

Pasame el caso 4 cuando lo tengas.

Usuario (Cristian Morales) 20/10/2026 10:32

Mantendría el criterio de línea de base: el test tiene que documentar que hoy el sistema acepta el email "@" y lo guarda, aunque debería rechazarlo.

```
def test_crear_estudiante_acepta_email_invalido(client):
    """Caso 4: el sistema acepta un email inválido."""
    datos = {
        "nombre": "Laura",
        "apellido": "Gomez",
        "legajo": "TEST-9004",
        "email": "@"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200 y lo crea. Esperado: 422 y no lo crea.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado == resultado
```

Claude 20/10/2026 10:32

Muy bien. El nombre `acepta_email_invalido` ya dice lo que pasa hoy, y el comentario deja claro lo esperado. Además verificás que quedó guardado, que es lo que pedía la línea de base.

Hay un solo ajuste, y es de fondo.

El dato inválido tiene que aparecer en el assert

`guardado == resultado` compara lo guardado con lo que respondió el sistema, pero no con lo que **vos mandaste**. Imaginá que en la Fase 3 alguien agrega `.strip()` o normaliza el email al guardar, y también al responder. Las dos cosas siguen siendo iguales entre sí, así que el test pasa, pero ya no guarda `"@"`. El comportamiento cambió y el test no lo detectó.

El objetivo de este test es mostrar que se guarda `"@"`. Que eso esté escrito explícitamente: comparalo contra lo que mandaste, no solo contra la respuesta.

Mantené también el `guardado == resultado`, porque cubre el resto de los campos.

Un detalle opcional

En el docstring podés sumar la referencia al hallazgo, por ejemplo `(defecto, hallazgo 4)`. Así el test, la tabla y la ficha quedan conectados, que es lo que te van a pedir mostrar en la defensa.

El caso 6 sigue el mismo patrón

Con `"apellido": None` es casi idéntico. Antes de copiar y pegar, una pregunta: hay dos tests que solo cambian en un campo y su valor. ¿Conviene dejarlos separados o usar `@pytest.mark.parametrize`? Pensá qué gana y qué pierde cada opción en legibilidad y en cómo se ve el reporte de pytest cuando algo falla. No hay una única respuesta correcta, pero tenés que poder justificar la que elijas.

Pasame el caso 4 ajustado y el 6, con tu decisión.

Usuario (Cristian Morales) 20/10/2026 10:35

no convendria dejarlos separados por ahora? ya que el caso 4 y 6 representan defectos distintos

te dejo el test corregido

```
def test_crear_estudiante_acepta_email_invalido(client):
    """Caso 4: acepta un email inválido (defecto, hallazgo 4)."""
    datos = {
        "nombre": "Laura",
        "apellido": "Gomez",
        "legajo": "TEST-9004",
        "email": "@"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200 y lo crea. Esperado: 422 y no lo crea.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado["email"] == datos["email"]
    assert guardado == resultado
```

y te paso el de apellido

```
def test_crear_estudiante_acepta_apellido_nulo(client):
    """Caso 6: acepta un apellido nulo (defecto, hallazgo 6)."""
    datos = {
        "nombre": "Martin",
        "apellido": None,
        "legajo": "TEST-9006",
        "email": "martin@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200 y lo crea. Esperado: 422 y no lo crea.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado["apellido"] == datos["apellido"]
    assert guardado == resultado
```

Claude 20/10/2026 10:35

▸ *Ejecutó un comando: Leer la tabla de hallazgos con sus casos de evidencia*

Sí, separados está bien justificado: son defectos distintos, y si en la Fase 3 corrigen uno y no el otro, cada test tiene que poder cambiar por su cuenta. Con `parametrize` los dos quedarían atados a la misma expectativa. Anotá esa razón, porque es una buena respuesta para la defensa.

Los dos tests están bien. Con `guardado["email"] == datos["email"]` el dato inválido queda explícito en el assert. El caso 6 sigue exactamente el mismo patrón, y que el `None` se guarde como `null` es justo lo que querés mostrar.

Un error en los docstrings: confundiste caso con hallazgo

Pusiste el mismo número de caso como número de hallazgo, pero en tu ficha de la Fase 1 no coinciden:

- **Hallazgo 6** es Primitive Obsession (áreas como strings). Su evidencia es el **caso 7**, no el 6.
- El **caso 6** (apellido nulo) es evidencia de los hallazgos **5** (falta de modelo de entrada) y **11** (integridad de datos).
- El **caso 4** (email `"@"`) es evidencia de los hallazgos **4** (validación de email duplicada) y **5**.

Revisá la columna de evidencia de tu tabla de smells y corregí los dos docstrings. La conexión entre test, caso y hallazgo es justo lo que te van a pedir mostrar, así que tiene que ser exacta. Si en la defensa alguien abre la ficha y no coincide, pierde credibilidad todo lo demás.

Revisá también el caso 10. Ahí no pusiste hallazgo, pero según la tabla corresponde a los hallazgos **3**, **4** y **5**. Agregarlo es opcional, pero mantiene la consistencia.

Corregí, corré `.\run.bat test` (tendrían que ser `6 passed`) y commiteá. Después seguimos con el caso 11, que es el más jugoso.

Usuario (Cristian Morales) 20/10/2026 10:37

ah bien tenes razon, te paso

Caso 4: acepta un email inválido (hallazgos 4 y 5).

Caso 6: acepta un apellido nulo (hallazgos 5 y 11).

Caso 10: email nulo provoca un error 500 (hallazgos 3, 4 y 5).

asi quedaria

Claude 20/10/2026 10:37

Así quedan correctos: los tres coinciden con la columna de evidencia de tu ficha.

Corré los tests y commiteá:

```
.\run.bat test
git add tests/test_estudiantes.py
git commit -m "test: casos 4 y 6, el sistema acepta datos invalidos (linea de base)"
git push
```

Tienen que ser `6 passed`.

Caso 11: legajo nulo dos veces

En la Fase 1 lo resolviste con tres pasos: dos POST con `"legajo": null` y un GET que mostraba los dos registros idénticos. Antes de escribirlo, pensá estas cuatro cosas:

1. **Qué querés probar exactamente.** No es solo que acepte `null`, porque eso ya lo muestra el caso 6 con el apellido. Lo nuevo de este caso es otra cosa. ¿Cuál? La respuesta define el nombre del test.
2. **Qué verificás después del segundo POST.** ¿Alcanza con que el segundo dé 200? ¿O tenés que comprobar algo sobre los **dos** registros juntos? Pensá cómo distinguir "creó dos" de "devolvió el mismo dos veces".
3. **El contraste con el caso 3.** Con un legajo normal repetido, el sistema sí lo detecta. Este test muestra el agujero de ese control. En la Fase 1 te expliqué la causa: `legajo = NULL` en SQL. Un comentario corto en el test que lo explique vale mucho.
4. **Los hallazgos.** Fijate en la ficha qué hallazgos lista el caso 11. Ojo, que esta vez separaste 11a y 11b.

¿Ya escribiste el test del caso 3 (legajo repetido)? Si no, conviene hacerlo antes que el 11: ese caso muestra que el control funciona, y el 11, dónde falla.

Usuario (Cristian Morales) 20/10/2026 10:37

me falto agregar los tests, me los pasas corregidos?

Claude 20/10/2026 10:38

Sí, son tus tests con los docstrings corregidos. No cambié nada más:

```
def test_crear_estudiante_acepta_email_invalido(client):
    """Caso 4: acepta un email inválido (hallazgos 4 y 5)."""
    datos = {
        "nombre": "Laura",
        "apellido": "Gomez",
        "legajo": "TEST-9004",
        "email": "@"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200 y lo crea. Esperado: 422 y no lo crea.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado["email"] == datos["email"]
    assert guardado == resultado


def test_crear_estudiante_acepta_apellido_nulo(client):
    """Caso 6: acepta un apellido nulo (hallazgos 5 y 11)."""
    datos = {
        "nombre": "Martin",
        "apellido": None,
        "legajo": "TEST-9006",
        "email": "martin@test.com"
    }

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200 y lo crea. Esperado: 422 y no lo crea.
    assert respuesta.status_code == 200

    resultado = respuesta.json()
    guardado = buscar_estudiante(client, resultado["id"])

    assert guardado is not None
    assert guardado["apellido"] == datos["apellido"]
    assert guardado == resultado


def test_crear_estudiante_email_null(client):
    """Caso 10: email nulo provoca un error 500 (hallazgos 3, 4 y 5)."""
    datos = {
        "nombre": "Pedro",
        "apellido": "Perez",
        "legajo": "TEST-9010",
        "email": None
    }

    cantidad_antes = cantidad_estudiantes(client)

    with TestClient(
        client.app,
        raise_server_exceptions=False
    ) as cliente_sin_excepciones:
        respuesta = cliente_sin_excepciones.post(
            "/estudiantes",
```

```
            json=datos
        )

    # Actual: 500. Esperado tras corregir el defecto: 422.
    assert respuesta.status_code == 500

    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

El caso 10 ya está en el archivo: reemplazá solo su docstring. Los casos 4 y 6 van a continuación, y el `from fastapi.testclient import TestClient` sigue arriba de todo.

Después: `.\run.bat test` (`6 passed`) y el commit que te pasé antes. Cuando esté, seguimos con el caso 3 y después el 11.

Usuario (Cristian Morales) 20/10/2026 10:39

perfecto quedo

Claude 20/10/2026 10:39

Bien, ya tenés seis tests en verde. Seguimos con el caso 3 y después el 11. Conviene hacerlos en ese orden porque se explican entre sí.

Caso 3: legajo repetido

En la Fase 1 mandaste el mismo body que en el caso 1 y el sistema respondió `200` con `{"error": "legajo repetido"}`. Según tu ficha, es evidencia del **hallazgo 3**.

Es un caso de error, igual que el 2, así que ya tenés el patrón. La diferencia está en la preparación:

1. **¿Qué tiene que existir antes del POST que querés probar?** Cada test arranca con una base nueva, así que no podés depender del caso 1. Tenés dos opciones: crear el estudiante dentro del test o usar uno de los datos de prueba que carga `main.py` (líneas 24–27). ¿Cuál elegirías y por qué? Pensá qué pasa con el test si en la Fase 3 sacás los datos de prueba del arranque (tu hallazgo 2).
2. **¿Dónde tomás `cantidad_antes`?** ¿Antes o después de crear el primer estudiante? Uno de los dos lugares hace que el test pruebe lo que querés y el otro no.
3. **El nombre.** Acá el sistema sí detecta el error, aunque con el código equivocado. El nombre tiene que reflejarlo, distinto de los "acepta_..." de los casos 4 y 6.

Pasámelo cuando lo tengas.

Usuario (Cristian Morales) 20/10/2026 10:40

se me ocurrio asi def test_crear_estudiante_con_legajo_repetido_devuelve_error(client): """Caso 3: detecta legajo repetido, pero responde con HTTP 200 (hallazgo 3).""" datos = { "nombre": "Juan", "apellido": "Perez", "legajo": "TEST-9011", "email": "juan.perez@test.com", "carrera": "Sistemas", }

```
# Primero creamos el estudiante que provocará el duplicado.
primera_respuesta = client.post("/estudiantes", json=datos)
assert primera_respuesta.status_code == 200
assert "error" not in primera_respuesta.json()

# La cantidad se mide después del primer registro.
cantidad_antes = cantidad_estudiantes(client)

# Intentamos registrar nuevamente el mismo legajo.
respuesta = client.post("/estudiantes", json=datos)

assert respuesta.status_code == 200
assert respuesta.json() == {"error": "legajo repetido"}

cantidad_despues = cantidad_estudiantes(client)
assert cantidad_despues == cantidad_antes
```

Claude 20/10/2026 10:40

▸ *Ejecutó un comando: Verificar el código esperado del caso 3*

Está muy bien resuelto. Respondiste bien las tres preguntas:

- **Creaste el estudiante dentro del test.** Así no depende de los datos de prueba, que en la Fase 3 vas a sacar del arranque.
- **Tomaste `cantidad_antes` después del primer POST.** Si lo midieras antes, el test esperaría que no cambie nada después de dos POST, y fallaría por el primero. Así medís solo el efecto del duplicado.
- **El nombre y el docstring** dicen lo que pasa: lo detecta, pero con el código equivocado.

También suma el `assert "error" not in ...` sobre el primer POST. Si la preparación falla, el test te lo dice ahí y no más abajo con un error confuso.

Hay dos detalles:

1. **Falta el comentario actual/esperado** en el assert del 200, y es el único test que no lo tiene. Según tu tabla de la Fase 1, el esperado es **409 Conflict**.
2. **Revisá el email en tu archivo.** En lo que me pasaste aparece como `[juan.perez@test.com](mailto:...)`. Seguramente se formateó como link al pegarlo en el chat, igual que se perdió la sangría, pero confirmá que en el `.py` sea el string simple `"juan.perez@test.com"`. Si quedó así, el test pasaría igual (tiene `@`), pero el dato no sería el que pensás.

Con eso, corré los tests (`7 passed`) y commiteá.

Ahora sí, el caso 11

Este test y el que viene se explican juntos: el caso 3 muestra que el control de duplicados funciona y el 11, dónde falla. Las preguntas que te dejé antes siguen en pie. La más importante es la 2: después del segundo POST, ¿cómo demostrás que hay **dos registros distintos** con `legajo: null`, y no uno solo devuelto dos veces?

Usuario (Cristian Morales) 20/10/2026 10:43

que te parece? primero confirmo que hay dos registros con legajo nulo, desdepues demuestro que son registros distintos, y comprueba que ambos post agregaron un estudiante

def test_crear_estudiante_acepta_legajo_nulo_repetido(client): """Caso 11: acepta dos estudiantes con legajo nulo (hallazgo 5)."""" datos = { "nombre": "Juan", "apellido": "Perez", "legajo": None, "email": "juan.test11@test.com", "carrera": "Sistemas", }

```
cantidad_antes = cantidad_estudiantes(client)

primera_respuesta = client.post("/estudiantes", json=datos)
segunda_respuesta = client.post("/estudiantes", json=datos)

assert primera_respuesta.status_code == 200
assert segunda_respuesta.status_code == 200

estudiantes = client.get("/estudiantes").json()
registros_nulos = [
    estudiante
    for estudiante in estudiantes
    if estudiante["legajo"] is None
]

# Deben existir dos registros diferentes, no una respuesta duplicada.
assert len(registros_nulos) == 2
assert registros_nulos[0]["id"] != registros_nulos[1]["id"]

cantidad_despues = cantidad_estudiantes(client)
assert cantidad_despues == cantidad_antes + 2
```

Claude 20/10/2026 10:43

Está bien pensado: los tres pasos que describís son los correctos, y el `cantidad_antes + 2` es lo que de verdad prueba que se crearon dos. Te marco cuatro cosas.

1. Una de las verificaciones no puede fallar nunca

`registros_nulos[0]["id"] != registros_nulos[1]["id"]` compara dos filas que salen de la base, y en la base `id` es la clave primaria. Dos filas distintas **siempre** tienen ids distintos, así que ese assert es verdadero por

definición: no aporta nada.

La pregunta que querías contestar ("¿creó dos o devolvió el mismo dos veces?") se responde con las **respuestas** de los dos POST, no con el listado. ¿Qué comparás entre `primera_respuesta` y `segunda_respuesta`? Y para conectarlas con la base ya tenés `buscar_estudiante`.

Con eso, el filtro de `registros_nulos` pasa a ser opcional. Si lo dejás, sirve para mostrar lo que viste en la captura 11c, pero depende de que los datos de prueba no tengan legajos nulos.

2. Los hallazgos están incompletos

Según tu ficha, el 11a es evidencia de los hallazgos **5 y 11**, y el 11b, del **11**. El 11 (integridad de datos) es justamente el central acá: sin `UNIQUE` ni `NOT NULL`, la base no frena nada. Tiene que figurar.

3. Faltan los dos comentarios

- **Actual/esperado**, como en todos los demás. Pensá qué debería pasar: ¿se rechaza el primero, el segundo o los dos?
- **Por qué falla el control.** Es lo que hace interesante este caso frente al 3: la consulta `WHERE legajo = ?` con `NULL` nunca da verdadero en SQL. Con una línea alcanza, y es lo primero que te van a preguntar en la defensa.

4. El email, otra vez

Volvió a aparecer como `[juan.test11@test.com](mailto:...)`. Si en el `.py` está bien, ignoralo: es el chat que lo convierte en link. Para que no te pase al pegar código, envolvelo entre triple backtick, como hiciste con la parte de abajo.

Ajustalo y pasámelo. Con este quedan cerrados todos los casos de estudiantes, y después vemos responsables (7, 8a, 8b).

Usuario (Cristian Morales) 31/10/2026 19:59

quiero retomar este trabajo
https://github.com/CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta/pull/4

deje aca el pr en draft, no recuerco como seguir

Claude 31/10/2026 20:00

Veo el PR para ver dónde quedaste.

▸ *Ejecutó un comando: Consultar el PR en draft (sin acceso)*

▸ *Ejecutó un comando: Consultar el PR por la API de GitHub (sin acceso)*

▸ *Pidió acceso de escritura al repositorio (el usuario lo rechazó)*

No pude abrir el PR: para leerlo necesito que me des acceso al repo, y rechazaste ese permiso. Está bien, no hace falta. Si querés que lo revise directamente, aceptalo cuando te lo vuelva a pedir. Si no, pasame lo que muestra el PR (los commits y los archivos cambiados) o la salida de `git log --oneline dev..test/personas`.

Igual, por lo que hablamos acá, quedaste en esto:

Hecho, en la rama `test/personas`:

- `pytest.ini` y `tests/conftest.py` provisorio: base aislada en una carpeta temporal por test.
- Captura `base_aislada.png` como evidencia del aislamiento.
- En `test_estudiantes.py`: alta válida, carrera por defecto y casos 2, 3, 4, 6 y 10, todos con el criterio de línea de base (comportamiento actual + comentario actual/esperado + hallazgos).

En revisión cuando cortamos: el caso 11 (legajo nulo dos veces). Tenías que hacer cuatro ajustes:

1. Cambiar el assert de ids del listado, que siempre da verdadero, por una comparación entre los ids de las **dos respuestas**, y conectarlos con la base mediante `buscar_estudiante`.
2. Hallazgos **5 y 11**, no solo el 5.
3. Agregar el comentario actual/esperado y una línea que explique por qué falla el control: `legajo = NULL` nunca da verdadero en SQL.
4. Revisar que el email en el `.py` sea un string simple.

Pendiente después:

- Los casos de responsables: 7, 8a y 8b.
- Avisarle a Bruno lo del `conftest.py`, si no lo hiciste.

- Pasar el PR de draft a listo **recién cuando esté puesto el tag** `etapa-diagnostico`, para que el diagnóstico y los tests no se mezclen.

Para saber cuál de los pendientes ya resolviste, ¿cerraste el caso 11 o arrancamos por ahí? ¿Y ya está mergeado el PR del diagnóstico y puesto el tag?

Usuario (Cristian Morales) 31/10/2026 20:00

si te doy permiso no quiero que hagas push

Claude 31/10/2026 20:01

Entendido: no voy a hacer push, ni commits, ni cambios en el PR. Pido acceso de solo lectura.

▸ *Obtuvo acceso de solo lectura al repositorio*

▸ *Ejecutó un comando: Clonar el repositorio en modo lectura y traer ramas, PRs y tags*

▸ *Ejecutó un comando: Revisar el historial de dev, los commits del PR #4 y los demás PRs*

▸ *Ejecutó un comando: Listar los tests del PR y la estructura del repositorio*

▸ *Ejecutó un comando: Correr los tests del PR*

Ya lo vi. Solo leí el repo: no hice push ni cambié nada.

Dónde estás

El PR #4 (`test/personas` → `dev`, en draft) tiene tus tres commits: base aislada, caso 10 y casos 4 y 6. Lo corrí y da `6 passed`, así que lo que está subido funciona.

Lo que falta en el PR:

- **El caso 3** (legajo repetido). Lo revisamos y quedó aprobado, pero no está commiteado. Le faltaba solo el comentario `# Actual: 200. Esperado: 409.`
- **El caso 11** (legajo nulo dos veces). Quedó con cuatro ajustes pendientes: comparar los ids de las **dos respuestas** en vez de los del listado, poner los hallazgos **5 y 11**, agregar el comentario actual/esperado más la explicación de `legajo = NULL`, y revisar el string del email.
- **Los casos de responsables** (7, 8a y 8b), que todavía no empezaste.

El estado del repo:

- Tu diagnóstico (PR #3) ya está mergeado en `dev`.
- **El tag `etapa-diagnostico` todavía no existe.** Solo está `estado-inicial`.
- El diagnóstico de Javier (PR #7) y la plantilla de la tabla de smells (PR #6) siguen abiertos. No veo ramas de diagnóstico de Gaston ni de Bruno.

Cómo seguir

1. **Primero, el código pendiente:** agregá el caso 3 con su comentario y cerrá el caso 11 con los ajustes. Si no lo tenés guardado localmente, reescribilo y pasámelo. Después commiteás vos.
2. **Después, responsables:** casos 7, 8a y 8b. Pensá si van en un `test_responsables.py` aparte. Probablemente sí.
3. **El tag:** `etapa-diagnostico` se pone cuando estén mergeados los diagnósticos de los cuatro. Preguntá en el grupo cómo vienen Gaston y Bruno. Mientras tanto, el PR sigue en draft.
4. **Bruno y el `conftest.py`:** si todavía no le avisaste, hacelo, porque sus tests van a depender de él.

¿Arrancamos con el caso 11? Pasame cómo lo tenés.

Usuario (Cristian Morales) 31/10/2026 20:02

me agregas los casos que faltan, me apsas los archivos que tengo que modificar?

Claude 31/10/2026 20:03

Los armo. Uso tu código del caso 3 (ya revisado) y aplico al 11 los ajustes que quedaron pendientes. Los de responsables son nuevos y siguen el mismo patrón que los tuyos.

▸ *Ejecutó un comando: Crear una copia local de trabajo y revisar los bodies de los casos*

▸ *Ejecutó un comando: Agregar los casos faltantes, correr los tests y preparar los archivos*

▸ *Envió los archivos test_estudiantes.py, test_responsables.py*

Agregué los casos que faltaban. Con eso la suite completa da `11 passed`. Todo quedó en una copia local: no toqué tu rama ni el PR.

`test_estudiantes.py` (reemplaza el tuyo): es el archivo del PR más dos tests al final.

- **Caso 3:** tu versión, con el comentario `Actual: 200. Esperado: 409 Conflict.` que le faltaba.
- **Caso 11:** tu versión con los cuatro ajustes. Ahora compara los ids de las dos respuestas, verifica cada registro con `buscar_estudiante`, cita los hallazgos 5 y 11, explica por qué `legajo = NULL` no detecta el duplicado y tiene el email como string simple.

`test_responsables.py` (nuevo): tiene sus propias auxiliares (`buscar_responsable` y `cantidad_responsables`) y tres casos:

- **Caso 7:** área inválida. Responde 200 con el error y no guarda nada (hallazgos 3 y 6).
- **Caso 8a:** alta válida. Verifica que quede `activo: True` y guardado (esperado: 201).
- **Caso 8b:** responsable duplicado. Crea dos registros distintos con los mismos datos (hallazgo 11; esperado: 409).

Para subirlo, copiá los dos archivos a `tests/` y corré:

```
.\run.bat test
git add tests/
git commit -m "test: casos 3 y 11 de estudiantes y casos de responsables (linea de base)"
git push
```

Dos cosas a tener en cuenta:

- **Los tests de responsables los escribí yo, no vos.** En `docs/uso-ia/cristian.md` agregá una fila para la Fase 2 que lo diga: los casos de estudiantes los escribiste vos con revisión de la IA, y los de responsables los generó la IA. Antes de la defensa, leelos y asegurate de poder explicarlos. Siguen tu mismo patrón, así que no deberías tener problema.
- **El PR sigue en draft** hasta que esté el tag `etapa-diagnostico`, y para eso faltan los diagnósticos de Gaston, Bruno y Javier (PR #7).

No voy a commitear ni pushear estos cambios: me pediste explícitamente que no hiciera push, y esta sesión tiene acceso de solo lectura al repo. Los cambios sin commitear están en una copia local de trabajo, no en tu rama. Los dos archivos ya te los mandé (`test_estudiantes.py` y `test_responsables.py`). El commit y el push los hacés vos con los comandos de mi mensaje anterior.

Usuario (Cristian Morales) 31/10/2026 20:06

como quedaria el archivo de la fase 2?

Claude 31/10/2026 20:07

▸ *Ejecutó un comando: Leer la declaración de uso de IA y los resultados de la Fase 1*

▸ *Ejecutó un comando: Leer el resto de la Fase 1 y listar los tests*

▸ *Ejecutó un comando: Redactar los resultados de la Fase 2 y actualizar la declaración de uso de IA*

▸ *Envió los archivos cristian_resultados_fase2.md, cristian.md*

Te armé el documento de resultados de la Fase 2 con el mismo formato que el de la Fase 1, y actualicé tu declaración de uso de IA. Igual que antes, no subí nada al repo.

`cristian_resultados_fase2.md` (va en `docs/pruebas/`, carpeta nueva; si el grupo acordó otra ubicación, movelo) explica:

- el criterio de línea de base y por qué separa el refactoring de la corrección de defectos;
- cómo `conftest.py` aísla la base, con la captura como evidencia;
- una tabla que vincula cada test con su caso de la Fase 1, el comportamiento actual, el esperado y los hallazgos;
- las decisiones técnicas: el `raise_server_exceptions` del caso 10, el `NULL` en SQL del caso 11 y por qué los casos 4 y 6 van separados.

`cristian.md` suma una fila de la Fase 2 y un párrafo en el detalle. Ahí queda claro qué escribiste vos (`conftest.py`, los tests de estudiantes y el criterio) y qué generó la IA (los tests de responsables, los ajustes finales del caso 11 y este documento). Revisá que coincida con cómo lo ves vos.

Dos cosas que quedaron a la vista al armarlo:

- **Faltan tests para los casos 5 (nombre nulo) y 12 (body como lista).** Figuran en el documento como pendientes. Son cortos y siguen el patrón que ya tenés, así que te conviene escribirlos vos: suman a tu parte en la defensa. Cuando los agregues, actualizá la tabla.

- **La evidencia de la fila de IA** apunta a `anexo/ia/cristian_claude_03_pruebas.pdf`, que todavía no existe. Es la exportación de esta conversación, como hiciste con las fases anteriores.

Sigo sin commitear ni pushear: me pediste que no hiciera push, y esta sesión solo tiene acceso de lectura al repo. Para que el aviso no se repita, guardo los cambios de mi copia local en un stash. Es reversible, y los archivos ya los tenés.

▸ *Ejecutó un comando: Guardar en un stash los cambios de la copia local (sin commit ni push)*

Listo: mi copia local quedó limpia, sin commits ni push. Tus cuatro archivos (`test_estudiantes.py`, `test_responsables.py`, `cristian_resultados_fase2.md` y `cristian.md`) ya te los mandé. El commit y el push en tu rama los hacés vos cuando los revises.

Usuario (Cristian Morales) 31/10/2026 20:10

donde irian estos ultimos archivvos?

Claude 31/10/2026 20:10

Los cuatro van en tu rama `test/personas`, en estas rutas:

Archivo	Ruta en el repo	Qué hacés
`test_estudiantes.py`	`tests/test_estudiantes.py`	Reemplaza el actual
`test_responsables.py`	`tests/test_responsables.py`	Nuevo
`cristian_resultados_fase2.md`	`docs/pruebas/cristian_resultados_fase2.md`	Nuevo: creá la carpeta `docs/pruebas/`
`cristian.md`	`docs/uso-ia/cristian.md`	Reemplaza el actual

Quedaría así:

```
tests/
  conftest.py
  test_estudiantes.py
  test_responsables.py
docs/
  diagnostico/   (Fase 1, ya está)
  pruebas/
    cristian_resultados_fase2.md
  uso-ia/
    cristian.md
  evidencias/fase2/cristian/base_aislada.png
```

Después:

```
git checkout test/personas
.\run.bat test
git add tests/ docs/
git status
git commit -m "test: casos 3, 11 y responsables; resultados de la fase 2"
git push
```

Tiene que dar `11 passed`. En el `git status` solo tendrían que aparecer esos cuatro archivos.

Sobre `docs/pruebas/`: la elegí para seguir el patrón de `docs/diagnostico/`. Si el grupo ya acordó otro lugar para los documentos de la Fase 2, por ejemplo en el README, usá ese.

Usuario (Cristian Morales) 31/10/2026 20:11

.\run.bat test
== test session starts ==
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 6 items

tests\test_estudiantes.py ... [50%]
tests\test_responsables.py ... [100%]

========== warnings summary ==========
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
========== 6 passed, 1 warning in 0.17s =

Claude 31/10/2026 20:11

No lo commitees todavía: algo no coincide. Tendrían que ser `11 passed` (8 de estudiantes y 3 de responsables), y te dio 6: **solo 3 de estudiantes**. Responsables está bien.

Tu `test_estudiantes.py` tiene solo los tres primeros tests (alta válida, carrera por defecto y sin apellido). Es la versión de tu primer commit. El archivo que te mandé tiene 8, así que no se reemplazó o quedó una versión vieja. Las causas probables:

1. **Estás en otra rama**, una que tiene solo el primer commit.
2. **El archivo descargado no pisó al viejo**, por ejemplo porque Windows lo guardó como `test_estudiantes (1).py`.

Para saber cuál es, corré:

```
git branch --show-current
git log --oneline -3
Get-ChildItem tests
```

- La rama tiene que ser `test/personas`, y el último commit, `5e953a0 test: casos 4 y 6...`.
- En `tests/` no tiene que haber archivos duplicados con `(1)`.

Pasame la salida. Lo que tenés que lograr es `11 passed` antes de commitear: si commiteás así, se pierden del repo los casos 4, 6 y 10, que ya estaban subidos.

Usuario (Cristian Morales) 31/10/2026 20:11

5e953a0 (HEAD -> test/personas, origin/test/personas) test: casos 4 y 6, el sistema acepta datos invalidos (linea de base)
2defe9a test: caso 10, email nulo provoca error 500 (linea de base)
e766aaa test: base aislada para tests y casos de alta de estudiantes
Get-ChildItem tests

Directorio: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\ms2\tpi\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\tests

Mode	LastWriteTime	Length	Name
d-----	31/10/2026 08:05 p. m.		__pycache__
-a----	20/10/2026 12:25 a. m.	882	conftest.py
-a----	31/10/2026 08:04 p. m.	2698	test_estudiantes.py
-a----	31/10/2026 08:04 p. m.	2698	test_responsables.py

Claude 31/10/2026 20:12

▸ *Ejecutó un comando: Comparar el tamaño de los archivos entregados*

Encontré el problema: **los dos archivos pesan exactamente lo mismo (2698 bytes)**. Pegaste el contenido de `test_responsables.py` también en `test_estudiantes.py`. Por eso pytest muestra 3 tests en cada uno: son los tres de responsables, repetidos. La rama y los commits están bien.

El `test_estudiantes.py` que te mandé tiene 8 tests y pesa unos 6,5 KB.

Para arreglarlo:

1. Abrí el `test_estudiantes.py` que te mandé (el de 8 tests, que empieza con `from fastapi.testclient import TestClient`) y pegá ese contenido en `tests/test_estudiantes.py`.
2. Verificá antes de correr los tests:

`powershell Get-ChildItem tests Select-String -Path tests\*.py -Pattern "^def test_" | Measure-Object`

El primer comando tiene que mostrar tamaños distintos (alrededor de 6,7 KB y 2,7 KB). El segundo tiene que contar **11**. 3. `.\run.bat test`: `11 passed`.

Y cuando hagas el `git add`, no incluyas la carpeta `__pycache__`. Fijate si el `.gitignore` la excluye: si en el `git status` aparece `tests/__pycache__/`, avisame antes de commitear.

Usuario (Cristian Morales) 31/10/2026 20:12

```
============================================================ test session starts
============================================================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1
collected 11 items

tests\test_estudiantes.py ........                                      [ 72%]
tests\test_responsables.py ...                                          [100%]

============================================================ warnings summary
============================================================
.venv\Lib\site-packages\fastapi\testclient.py:1

C:\Users\crist\OneDrive\Escritorio\Cuarto_cuatri\MS2\TPI\TPI_MS2_Mazza_Morales_Ovelar_Paschetta\.venv\Lib\
site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using `httpx` with `starlette.testclient` is
deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
============================================================ 11 passed, 1 warning in 0.30s
===
```

ahora si

Claude 31/10/2026 20:12

Ahora sí: `11 passed`, con 8 tests de estudiantes y 3 de responsables.

Antes de commitear, confirmá que los dos documentos estén en su lugar y que no se cuele `__pycache__`:

```
git status
```

Tendrían que aparecer solo estos cuatro:

```
modified:   docs/uso-ia/cristian.md
modified:   tests/test_estudiantes.py
untracked:  docs/pruebas/
untracked:  tests/test_responsables.py
```

Si está así:

```
git add tests/test_estudiantes.py tests/test_responsables.py docs/pruebas/ docs/uso-ia/cristian.md
git commit -m "test: casos 3, 11 y responsables; resultados de la fase 2"
git push
```

Agrego los archivos por nombre a propósito, para que `__pycache__` no entre aunque el `.gitignore` no lo excluya. Si en el `status` aparece, sumá `__pycache__/` al `.gitignore` en un commit aparte.

Después del push, el PR #4 se actualiza solo. Lo que te queda de la Fase 2:

- escribir vos los casos 5 y 12, y actualizar la tabla del documento;
- exportar esta conversación como `anexo/ia/cristian_claude_03_pruebas.pdf`;
- pasar el PR de draft a listo cuando esté el tag `etapa-diagnostico`.

Usuario (Cristian Morales) 31/10/2026 20:13

git status
On branch test/personas
Your branch is up to date with 'origin/test/personas'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified: tests/test_estudiantes.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        tests/test_responsables.py

no changes added to commit (use "git add" and/or "git commit -a")

Claude 31/10/2026 20:13

Los tests están bien, pero **faltan los dos documentos**: no aparecen ni `docs/pruebas/` ni la modificación de `docs/uso-ia/cristian.md`. Todavía no los copiaste al repo.

Te conviene separarlo en dos commits: uno con los tests y otro con la documentación. Así el historial queda más claro.

1. Commiteá los tests ahora:

```
git add tests/test_estudiantes.py tests/test_responsables.py
git commit -m "test: casos 3 y 11 de estudiantes y casos de responsables (linea de base)"
```

2. Copiá los documentos:

- Creá la carpeta `docs\pruebas\` y poné ahí `cristian_resultados_fase2.md`.
- Reemplazá `docs\uso-ia\cristian.md` por el que te mandé.

Antes de commitear, abrí el `cristian.md` y fijate que tenga la fila de **Pruebas (Fase 2)**. Así evitás que se repita lo de recién.

3. Commiteá la documentación y subí todo:

```
git status
git add docs/pruebas/ docs/uso-ia/cristian.md
git commit -m "docs: resultados de la fase 2 y registro de uso de IA"
git push
```

En el `git status` del paso 3 tendrían que aparecer solo esos dos documentos.

Usuario (Cristian Morales) 31/10/2026 20:13

pasamelos de nuevo

Claude 31/10/2026 20:13

▸ *Envió los archivos cristian_resultados_fase2.md, cristian.md*

Te los mandé de nuevo:

- `cristian_resultados_fase2.md` va en `docs/pruebas/`. La carpeta es nueva, así que creala.
- `cristian.md` reemplaza al que está en `docs/uso-ia/`.

Antes de commitear, fijate que cada archivo quedó en su lugar. Así no te pasa lo mismo que con los tests:

```
(Get-Item docs\pruebas\cristian_resultados_fase2.md).Length
Select-String -Path docs\uso-ia\cristian.md -Pattern "Pruebas \(Fase 2\)"
```

El primer comando tiene que mostrar un número de alrededor de 5.000. El segundo tiene que encontrar la fila nueva. Si están bien, seguí con el paso 3 que te pasé antes.

Usuario (Cristian Morales) 31/10/2026 20:14

git status
On branch test/personas
Your branch is up to date with 'origin/test/personas'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified: docs/uso-ia/cristian.md
        modified: tests/test_estudiantes.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        docs/pruebas/
        tests/test_responsables.py

no changes added to commit (use "git add" and/or "git commit -a")

Claude 31/10/2026 20:14

Ahora están los cuatro archivos. Todavía no commiteaste los tests, así que hacé los dos commits seguidos:

```
git add tests/test_estudiantes.py tests/test_responsables.py
git commit -m "test: casos 3 y 11 de estudiantes y casos de responsables (linea de base)"

git add docs/pruebas/ docs/uso-ia/cristian.md
git commit -m "docs: resultados de la fase 2 y registro de uso de IA"

git push
```

Para confirmar que quedó bien:

```
git log --oneline -5
git status
```

El log tiene que mostrar tus dos commits nuevos arriba de `5e953a0`, y el status tiene que decir `nothing to commit, working tree clean`. Si es así, el PR #4 ya tiene todo.

Usuario (Cristian Morales) 31/10/2026 20:14

76bb362 (HEAD -> test/personas, origin/test/personas) docs: resultados de la fase 2 y registro de uso de IA
db537d6 test: casos 3 y 11 de estudiantes y casos de responsables (linea de base)
5e953a0 test: casos 4 y 6, el sistema acepta datos invalidos (linea de base)
2defe9a test: caso 10, email nulo provoca error 500 (linea de base)
e766aaa test: base aislada para tests y casos de alta de estudiantes

Claude 31/10/2026 20:15

▸ *Ejecutó un comando: Verificar la rama subida*

▸ *Ejecutó un comando: Verificar el último commit de la rama en GitHub*

▸ *Ejecutó un comando: Correr los tests sobre la rama subida*

Los dos commits están subidos y el PR #4 ya los incluye. Bajé la rama desde GitHub y la corrí en limpio: da `11 passed`. Tiene los 7 archivos esperados y no se coló ningún `__pycache__`.

Lo que te queda para cerrar la Fase 2:

1. **Escribir los tests de los casos 5 y 12.** Son cortos y siguen tu patrón. Después cambiá la sección "Casos sin test todavía" del documento por dos filas en la tabla.
2. **Exportar esta conversación** como `anexo/ia/cristian_claude_03_pruebas.pdf`. La fila de uso de IA ya apunta a ese archivo.
3. **Avisarle a Bruno** que el `conftest.py` está en `test/personas`, si todavía no lo hiciste.
4. **Pasar el PR de draft a listo** cuando esté el tag `etapa-diagnostico`. Para eso faltan los diagnósticos de Gaston y Bruno, y que se mergee el de Javier (PR #7).

Si querés, seguimos con el caso 5: pasámelo cuando lo tengas y lo revisamos.

Usuario (Cristian Morales) 31/10/2026 20:15

cuales eran los casos 5 y 12?

Claude 31/10/2026 20:16

Según tu tabla de la Fase 1:

Caso 5: nombre nulo

- **Body:** `{"nombre": null, ...}` con el resto de los campos válidos.
- **Hoy:** `200` con `{"error": "falta nombre"}`, y no lo crea.
- **Esperado:** `422`.
- **Hallazgo:** 3 (manejo de errores inconsistente).

Fijate en el contraste con el caso 6: con `apellido: null` el sistema lo acepta, y con `nombre: null` lo rechaza. La diferencia está en la línea 61, que chequea `is None`, mientras que la 63 no lo hace. Por eso tu Fase 1 dice que "el comportamiento cambia según el campo".

Caso 12: body que no es un objeto

- **Body:** `[1, 2]`, una lista en lugar de un diccionario.
- **Hoy:** `422` con `"Input should be a valid dictionary"` dentro de `detail`. Esta validación la hace FastAPI, no el código de `main.py`.
- **Esperado:** `422` con el mismo formato que el resto de los errores. El código es correcto, pero el formato no: los demás usan `error` y este usa `detail`.
- **Hallazgos:** 3 y 5.

Es el único caso donde el código HTTP ya es el correcto. Lo que el test tiene que dejar registrado es el **formato** de la respuesta.

Antes de escribirlos, pensá:

- **Caso 5:** ya tenés el patrón del caso 2. ¿Qué cambia?
- **Caso 12:** el contenido de `detail` es largo y lo arma FastAPI (o en realidad Pydantic). Si comparás la respuesta completa con `==`, el test puede romperse con solo actualizar la librería, aunque el sistema no haya cambiado. ¿Qué parte de la respuesta te interesa verificar para mostrar el problema del formato, sin atarte al texto exacto del mensaje?

Escribí el 5 primero y pasámelo.

Usuario (Cristian Morales) 31/10/2026 20:17

seria algo asi?

```
def test_crear_estudiante_sin_nombre(client):
    """Caso 2: falta nombre."""
    datos = {"apellido": "Perez", "legajo": "TEST-9003", "email": "pedro@test.com"}

    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200. Esperado según el diagnóstico: 422.
    assert respuesta.status_code == 200
    assert respuesta.json() == {"error": "falta nombre"}
```

```
    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

Claude 31/10/2026 20:17

Va bien: el patrón es el correcto, con la cantidad antes y después y el comentario actual/esperado. Pero el test prueba otro caso. Hay tres ajustes:

1. No es el caso 5 (lo más importante)

El caso 5 es **nombre nulo**: el campo se manda, pero vale `null`. Vos sacaste el campo del body, así que estás probando "falta nombre", que es otro camino.

Mirá la línea 61:

```
if "nombre" not in data or data["nombre"] == "" or data["nombre"] is None:
```

Tu test pasa por la primera condición (`not in data`). El caso 5 pasa por la tercera (`is None`). Las dos respuestas son iguales hoy, pero si en la Fase 3 alguien rompe el chequeo de `None`, tu test no lo detectaría.

Y justo esa condición es la que importa: es lo que hace que `nombre: null` se rechace y `apellido: null` (caso 6) se acepte. Mandá el campo con `None`, igual que hiciste en el caso 6 con el apellido.

2. El nombre y el docstring

- El docstring dice **"Caso 2"**. Tiene que decir **Caso 5** y citar el **hallazgo 3**, como los demás.
- `test_crear_estudiante_sin_nombre` describe la ausencia del campo. Ponele un nombre que diga "nulo", en línea con `acepta_apellido_nulo`, pero que muestre que este sí se rechaza.

3. El legajo

`TEST-9003` ya lo usa el caso 2. No rompe nada, porque cada test tiene su propia base, pero venías usando el número del caso como sufijo. Para mantener la trazabilidad, usá `TEST-9005`.

Ajustalo y pasámelo. Después vamos con el 12.

Usuario (Cristian Morales) 31/10/2026 20:20

```
def test_crear_estudiante_nombre_nulo(client):
    """Caso 5: falta nombre."""
    datos = {"nombre": None, "apellido": "Perez", "legajo": "TEST-9003", "email": "pedro@test.com"}

    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200. Esperado según el diagnóstico: 422.
    assert respuesta.status_code == 200
    assert respuesta.json() == {"error": "falta nombre"}

    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes

ahi?
```

Claude 31/10/2026 20:20

Sí, ya es el caso 5: ahora el test pasa por la condición `is None` de la línea 61, que es lo que había que probar. La estructura y el comentario están bien.

Quedan dos detalles de los que te marqué:

1. **El docstring.** Dice "falta nombre", que describe el caso de un campo ausente. Además le falta el hallazgo. Algo así: `"""Caso 5: rechaza un nombre nulo, pero con HTTP 200 (hallazgo 3)."""`
2. **El legajo** sigue siendo `TEST-9003`. Cambialo a `TEST-9005`.

Opcional, pero suma: un comentario corto que marque el contraste con el caso 6. Por ejemplo, que `nombre: null` se rechaza porque la línea 61 chequea `is None`, mientras que `apellido: null` se acepta porque la línea 63

no lo hace. Es la inconsistencia que tu Fase 1 señala ("el comportamiento cambia según el campo"), y así queda a la vista en el código.

El nombre `nombre_nulo` sirve. Si querés que sea simétrico con `acepta_apellido_nulo`, podría ser `rechaza_nombre_nulo`, pero es una cuestión de gusto.

Con eso queda cerrado. Pegalo después del caso 4, para respetar el orden de los casos, o al final del archivo. Corré `.\run.bat test` (tendrían que ser `12 passed`) y seguimos con el caso 12, que es el más interesante de los dos.

Usuario (Cristian Morales) 31/10/2026 20:21

```
def test_crear_estudiante_nombre_nulo(client):
    """Caso 5: rechaza un nombre nulo, pero con HTTP 200 (hallazgo 3)."""
    datos = {"nombre": None, "apellido": "Perez", "legajo": "TEST-9005", "email": "pedro@test.com"}

    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=datos)

    # Actual: 200. Esperado según el diagnóstico: 422.
    assert respuesta.status_code == 200
    assert respuesta.json() == {"error": "falta nombre"}

    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

Claude 31/10/2026 20:21

Quedó cerrado: el docstring y el legajo ya están corregidos. Es tu test, escrito con el mismo patrón que los demás. Pegalo en `tests/test_estudiantes.py`, corré `.\run.bat test` y confirmá que da `12 passed`.

Caso 12: body como lista

Mandás `[1, 2]` y hoy responde `422`. El código es correcto. Lo que está mal es el formato: esta respuesta usa `detail` y el resto de los errores usa `error`. Eso es lo que el test tiene que registrar.

Antes de escribirlo, pensá estas tres cosas:

1. **Qué verificás de la respuesta.** El contenido de `detail` lo arma Pydantic y es largo: incluye tipo, ubicación, mensaje y la URL de la documentación. Si lo comparás entero con `==`, el test se rompe al actualizar la librería aunque `main.py` no haya cambiado. ¿Qué te alcanza para mostrar el problema? Pensá en las **claves** de la respuesta, no en el texto.
2. **Cómo mandás el body.** Hasta ahora usaste `json=datos` con un diccionario. ¿Funciona igual con una lista?
3. **El comentario actual/esperado.** Acá el código no cambia. ¿Qué escribirías como "esperado"?

Fijate también: en este caso no hace falta la cantidad antes y después para saber si se guardó algo, porque el request ni siquiera llega a la función. Aun así, ¿la dejarías por consistencia? Cualquiera de las dos opciones está bien, pero tenés que poder justificarla.

Pasámelo cuando lo tengas.

Usuario (Cristian Morales) 31/10/2026 20:25

```
def test_caso_12_body_como_lista(client):
    # Actual: responde 422 con "detail" (formato de FastAPI).
    # Esperado: responde 422 manteniendo el formato "error" de la API.

    response = client.post("/estudiantes", json=[1, 2])

    assert response.status_code == 422
    assert "detail" in response.json()
    assert "error" not in response.json()
```

Claude 31/10/2026 20:25

Lo central está muy bien: verificás las **claves** de la respuesta, no el texto de Pydantic. Así el test no se rompe al actualizar la librería y muestra exactamente el problema, que hay dos formatos de error distintos. `json=[1, 2]` también funciona: `json=` acepta cualquier valor serializable. El comentario actual/esperado es claro.

Lo que falta es ajustarlo al estilo del resto del archivo:

1. **El nombre.** `test_caso_12_...` no sigue el patrón de los demás (`test_crear_estudiante_...`, que dice qué pasa). Por ejemplo: `test_crear_estudiante_body_lista_responde_otro_formato`. El número de caso va en el docstring.
2. **Falta el docstring.** Es el único test sin uno. Tiene que tener el caso y los hallazgos: **3 y 5**.
3. **`response` en vez de `respuesta`.** Todo el archivo está en castellano. Mantené la consistencia, que es justamente lo que marcaste en tu hallazgo 12 (Inconsistent Naming).

La decisión sobre la cantidad

La sacaste, y es válido, pero no me dijiste por qué. Pensalo así: hoy el request ni llega a `crear_estudiante`, porque lo frena FastAPI. En la Fase 3, cuando resuelvas el hallazgo 5 con un modelo de entrada, cambia quién valida. ¿Querés que el test te avise si en ese cambio algo termina guardándose?

Si la dejás, anotá en el documento que es por protección y consistencia. Si no, dejá un comentario corto en el test que explique por qué no hace falta. Las dos opciones están bien, pero tiene que quedar escrita la razón.

Ajustalo y pasámelo. Con este quedan cubiertos todos tus casos de la Fase 1.

Usuario (Cristian Morales) 31/10/2026 20:26

como pondriasn el docstring?

Claude 31/10/2026 20:26

El mismo formato que los demás: caso, qué pasa hoy y hallazgos:

```
"""Caso 12: body que no es un objeto responde 422 con otro formato de error (hallazgos 3 y 5)."""
```

Si te queda largo, podés hacerlo más corto:

```
"""Caso 12: body como lista, 422 con formato "detail" en vez de "error" (hallazgos 3 y 5)."""
```

Con cualquiera de los dos se entiende que el código es correcto y que el problema es el formato.

Usuario (Cristian Morales) 31/10/2026 20:26

```
def test_caso_12_body_como_lista(client):
    """Caso 12: body como lista, 422 con formato "detail" en vez de "error" (hallazgos 3 y 5)."""

    respuesta = client.post("/estudiantes", json=[1, 2])

    assert respuesta.status_code == 422
    assert "detail" in respuesta.json()
    assert "error" not in respuesta.json()

que te parece?
```

Claude 31/10/2026 20:26

Va bien: el docstring y `respuesta` quedaron como el resto. Te faltan tres cosas, y una la tenías y se perdió:

1. **Desapareció el comentario actual/esperado.** En la versión anterior estaba y era bueno. Volvé a ponerlo antes de los asserts. Sin él, este es el único test que no dice qué debería pasar.
2. **El nombre sigue siendo `test_caso_12_body_como_lista`.** Cambialo al estilo del archivo, por ejemplo `test_crear_estudiante_body_lista_responde_otro_formato`.
3. **La decisión sobre la cantidad.** Sigue sin estar, y no hay comentario que explique por qué. Elegí una de las dos: - la agregás, como en el caso 2, por protección para la Fase 3; - o dejás una línea que diga que no hace falta porque FastAPI rechaza el body antes de llegar a `crear_estudiante`.

Con esos tres cambios, queda cerrado.

Usuario (Cristian Morales) 31/10/2026 20:27

como ajustarias la decision de la cantidad?

Claude 31/10/2026 20:27

Yo la dejaría. Tiene un costo mínimo (dos líneas) y te protege justo en el momento en que más falta hace: en la Fase 3, cuando resuelvas el hallazgo 5 y cambie quién valida el body, el test te avisa si algo termina guardándose. Además queda igual que el resto de los casos de error.

En tu test quedaría así:

```
    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=[1, 2])

    # Actual: 422 con "detail" (lo genera FastAPI).
    # Esperado: 422 con el formato "error" del resto de la API.
    assert respuesta.status_code == 422
    assert "detail" in respuesta.json()
    assert "error" not in respuesta.json()

    # Hoy FastAPI rechaza el body antes de llegar a crear_estudiante.
    # Se verifica igual por si cambia la validación en la Fase 3.
    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

El comentario de abajo es tu justificación por escrito: dice por qué hoy no podría guardarse nada y por qué igual lo verificás. Si en la defensa te preguntan, es esa misma respuesta.

Cambiale el nombre al test, corré `.\run.bat test` (tendrían que ser `13 passed`) y pasamos a actualizar el documento.

Usuario (Cristian Morales) 31/10/2026 20:28

```
def test_crear_estudiante_body_lista_responde_otro_formato(client):
    """Caso 12: body como lista, 422 con formato "detail" en vez de "error" (hallazgos 3 y 5)."""
    # Actual: responde 422 con "detail" (formato de FastAPI).
    # Esperado: responde 422 manteniendo el formato "error" de la API.

    respuesta = client.post("/estudiantes", json=[1, 2])

    # Hoy FastAPI rechaza el body antes de llegar a crear_estudiante.
    # Se verifica igual por si cambia la validación en la Fase 3.
    assert respuesta.status_code == 422
    assert "detail" in respuesta.json()
    assert "error" not in respuesta.json()
```

ahi quedaria

Claude 31/10/2026 20:28

Casi. Copiaste el comentario, pero **faltan las dos líneas que verifica**: `cantidad_antes` y el assert de `cantidad_despues`. Ahora el comentario dice "se verifica igual", pero el test no verifica nada de eso. Es peor que no tener el comentario, porque quien lo lea va a creer que está cubierto.

Agregá:

- `cantidad_antes = cantidad_estudiantes(client)` **antes** del POST;
- al final, `cantidad_despues = cantidad_estudiantes(client)` y `assert cantidad_despues == cantidad_antes`.

Y moví el comentario "Hoy FastAPI rechaza..." justo arriba de esas dos últimas líneas, que son las que explica. Ahora está arriba de los asserts del 422, que no tienen nada que ver.

El nombre y el resto ya están bien. Con ese ajuste, `13 passed`.

Usuario (Cristian Morales) 31/10/2026 20:29

me quedo asi

```
def test_crear_estudiante_body_lista_responde_otro_formato(client):
    """Caso 12: body como lista, 422 con formato "detail" en vez de "error" (hallazgos 3 y 5)."""
    # Actual: responde 422 con "detail" (formato de FastAPI).
    # Esperado: responde 422 manteniendo el formato "error" de la API.

    cantidad_antes = cantidad_estudiantes(client)

    respuesta = client.post("/estudiantes", json=[1, 2])

    assert respuesta.status_code == 422
    assert "detail" in respuesta.json()
    assert "error" not in respuesta.json()

    # Hoy FastAPI rechaza el body antes de llegar a crear_estudiante.
    # Se verifica igual por si cambia la validación en la Fase 3.
    cantidad_despues = cantidad_estudiantes(client)
    assert cantidad_despues == cantidad_antes
```

Claude 31/10/2026 20:29

Ahora sí, quedó cerrado: el comentario explica justo las líneas que tiene abajo, y el test es consistente con todos los demás. Con este, tus 13 casos de la Fase 1 tienen test.

Pegalo en `tests/test_estudiantes.py` junto con el caso 5 y corré `.\run.bat test`. Tiene que dar `13 passed`.

Actualizar el documento

En `docs/pruebas/cristian_resultados_fase2.md` hay tres cambios:

1. **Arriba:** `11 tests, 11 en verde` pasa a `13 tests, 13 en verde`.
2. **En la tabla "Tests y casos":** agregá dos filas en su lugar. La del caso 5 va después del 4, y la del 12, al final:

`| 5 | \`test_crear_estudiante_nombre_nulo\` | 200 con error, no lo crea | 422 | 3 | | 12 | \`test_crear_estudiante_body_lista_responde_otro_formato\` | 422 con formato \`detail\` | 422 con formato \`error\` | 3, 5 |`

3. **Borrá la sección "Casos sin test todavía"** completa, con su tabla.

Si querés, sumá en "Detalles técnicos" una línea sobre el caso 12: verifica las claves de la respuesta y no el texto, para no depender de la versión de Pydantic. Es una decisión tuya y vale tenerla escrita.

Después:

```
git add tests/test_estudiantes.py docs/pruebas/cristian_resultados_fase2.md
git commit -m "test: casos 5 y 12 de estudiantes (linea de base)"
git push
```

Usuario (Cristian Morales) 31/10/2026 20:31

pasame el md de fase 2 ya con las modificaciones

Claude 31/10/2026 20:31

▸ *Ejecutó un comando: Actualizar el documento de la Fase 2 con los casos 5 y 12*

▸ *Envió el archivo cristian_resultados_fase2.md*

Actualicé `cristian_resultados_fase2.md`, que reemplaza al de `docs/pruebas/`. Tiene estos cambios:

- arriba dice `13 tests, 13 en verde`;
- la tabla suma las filas de los casos 5 y 12 en su orden;
- saqué la sección "Casos sin test todavía";
- en "Detalles técnicos" agregué dos puntos: por qué el caso 5 manda `null` y no omite el campo (el contraste con el caso 6), y por qué el caso 12 verifica las claves y no el texto, y también la cantidad.

Para subirlo, primero corré `.\run.bat test`. Tiene que dar `13 passed`. Si da eso:

```
git add tests/test_estudiantes.py docs/pruebas/cristian_resultados_fase2.md
git commit -m "test: casos 5 y 12 de estudiantes (linea de base)"
git push
```

Esta vez los tests de los casos 5 y 12 los escribiste vos. El documento lo actualicé yo, y eso ya figura en tu declaración de uso de IA.

Usuario (Cristian Morales) 31/10/2026 20:32

ahi actualice la branch, queres revisar?

Claude 31/10/2026 20:33

▸ *Ejecutó un comando: Traer la rama actualizada, correr los tests y revisar los casos nuevos*

▸ *Ejecutó un comando: Verificar la fila de uso de IA*

Lo revisé: está todo bien.

- **El commit** `476fc88` está subido arriba de los anteriores.
- **Los tests:** bajé la rama desde GitHub y da `13 passed`. Los casos 5 y 12 están como los dejamos: el 5 manda `nombre: None` con `TEST-9005`, y el 12 tiene el comentario actual/esperado, la verificación de claves y la cantidad con su justificación.
- **El documento** coincide con el que te mandé. Solo le falta el salto de línea final, que no importa.
- **No se coló ningún** `__pycache__`**.**

Hay un detalle que quedó desactualizado: en `docs/uso-ia/cristian.md`, la fila de la Fase 2 dice que escribiste los tests de los casos "*1, 2, 3, 4, 6, 10 y 11*". Ahora también escribiste el 5 y el 12. Cambialo a **"1 a 6, 10, 11 y 12"** y commitealo:

```
git add docs/uso-ia/cristian.md
git commit -m "docs: actualizar registro de uso de IA con casos 5 y 12"
git push
```

Con eso, tu parte de la Fase 2 está completa. Lo que queda depende del grupo:

- exportar esta conversación como `anexo/ia/cristian_claude_03_pruebas.pdf`;
- avisarle a Bruno lo del `conftest.py`;
- poner el tag `etapa-diagnostico` cuando estén mergeados los diagnósticos de los cuatro, y recién ahí pasar el PR #4 de draft a listo.

Usuario (Cristian Morales) 31/10/2026 20:34

pasame como quedaria el .md

Claude 31/10/2026 20:34

▸ *Ejecutó un comando: Actualizar la declaración de uso de IA con los casos 5 y 12*

▸ *Envió el archivo cristian.md*

Te mandé `cristian.md` actualizado: reemplazá el de `docs/uso-ia/`. Partí de la versión que está en tu rama y cambié solo una cosa. En la fila de la Fase 2, donde decía "*casos 1, 2, 3, 4, 6, 10 y 11*", ahora dice **"casos 1 a 6, 10, 11 y 12"**.

Después:

```
git add docs/uso-ia/cristian.md
git commit -m "docs: actualizar registro de uso de IA con casos 5 y 12"
git push
```